TECHNISCHE UNIVERSITÄT
CHEMNITZ

Privacy-Preserving Source Code Vulnerability Detection and Repair using Retrieval-Augmented LLMs for Visual Studio Code

Master Thesis

Submitted in Fulfilment of the
Requirements for the Academic Degree
M.Sc. Web Engineering

Dept. of Computer Science
Chair of Computer Engineering

Submitted by: Md Hafizur Rahman
Student ID: 806286
Date: 15.12.2025
Internal Supervisor: Abubaker Gaber M.Sc.
Internal Examiner: Dr.-Ing. Sebastian Heil

Abstract

Developers want security feedback directly in the IDE while they write code. Traditional static analysis tools work well for many rule-based problems, but they often miss issues that need deeper reasoning or security context. Large Language Models (LLMs) may help with vulnerability detection and repair, but they also bring problems such as unstable quality, many false positives, and privacy risks when source code is sent to cloud services.

This thesis studies a privacy-preserving way to detect and repair vulnerabilities by combining locally running LLMs with Retrieval-Augmented Generation (RAG) in a VS Code extension. The system runs on the developer's device and uses locally stored CWE, CVE, and OWASP knowledge, so source code does not need to leave the machine. It provides two modes: a low-latency inline mode for IDE-triggered feedback, and an audit mode that adds retrieved security knowledge to the model's analysis.

The system is evaluated on 128 JavaScript/TypeScript cases (113 vulnerable and 15 secure) based on CWE/OWASP-related concepts. The evaluation uses precision, recall, F1-score, false-positive rate, and latency. The results show trade-offs between speed, quality, and noise that depend on the model and prompting mode. The best F1-score (63.76%) is reached by `qwen3:8b` with RAG. Overall, the study shows that local privacy-preserving LLM workflows can support useful vulnerability analysis, but careful model choice and false-positive control are still necessary for practical use.

Keywords: source code security, vulnerability detection, secure code repair, LLMs, RAG, privacy-preserving systems, VS Code

Acknowledgment

I am grateful to everyone who supported me during this Master's thesis.

My sincere thanks go to my internal supervisor, *Abubaker Gaber, M.Sc.*, for his continuous guidance, constructive feedback, and thoughtful discussions throughout the project. His support was essential in shaping both the technical direction and the academic quality of this work.

I also thank the academic staff of the Master's program for providing a strong foundation and a stimulating learning environment.

I am equally thankful to my colleagues and peers for their encouragement and for creating a supportive atmosphere during the study period.

Most of all, I thank my family for their patience, trust, and constant encouragement. Their support made it possible for me to complete this thesis.

Task Description

Traditional rule-based security analysis tools are effective for detecting predefined vulnerability patterns but can struggle with context-dependent weaknesses that require semantic reasoning. Cloud-based language model assistants provide stronger contextual analysis, yet compromise confidentiality and reproducibility because proprietary source code must leave the local environment. This thesis designs and evaluates a fully local, privacy-preserving secure coding assistant for Visual Studio Code that integrates large language models (LLMs) with Retrieval-Augmented Generation (RAG). The prototype is built around a no-source-code-exfiltration boundary: analyzed code and IDE-derived context are processed on-device and sent only to a local LLM runtime.

The assistant combines two complementary modes: real-time inline diagnostics that surface debounced vulnerability alerts during active development, and asynchronous audit and question–answering modes for deeper inspection. In audit mode, the prompt can optionally be grounded with a local retrieval index over security guidance (e.g., CWE/OWASP-aligned summaries and optionally cached public CVE/NVD metadata). In question–answering mode, users provide explicit file/folder context and receive narrative security review guidance. A modular architecture separates the LLM inference layer, the retrieval pipeline, and the Visual Studio Code extension interface. Privacy is enforced primarily through local deployment (no external inference APIs) and by limiting knowledge refresh to public metadata; rerunability is supported by version-pinned artifacts where available and by explicit reporting of model fingerprints and runtime settings. The threat model includes prompt injection via attacker-controlled code text and retrieval-quality risks; the prototype mitigates these risks primarily through strict JSON output contracts in diagnostic workflows, defensive parsing, and curated retrieval sources, while stronger provenance controls remain future work.

Evaluation is conducted on a project-authored JavaScript/TypeScript suite aligned to CWE/OWASP concepts, combining vulnerable and secure/negative cases to quantify both detection quality and false-positive behavior. The comparison covers LLM-only and LLM+RAG prompting, and includes baseline SAST tools (Semgrep, CodeQL, and ESLint security rules) to contextualize trade-offs.

Evaluation metrics include precision, recall, F1-score, secure-sample false-positive rate, JSON parse success, and latency. Privacy and reproducibility are assessed primarily through architectural/configuration evidence (local inference boundary, optional refresh behavior) and run-provenance reporting under controlled local execution.

Contents

List of Figures

List of Tables

List of Abbreviations

AST	Abstract Syntax Tree
CVE	Common Vulnerabilities and Exposures
CWE	Common Weakness Enumeration
F1	F1 Score
FPR	False Positive Rate
HNSW	Hierarchical Navigable Small World
IDE	Integrated Development Environment
LLM	Large Language Model
LRU	Least Recently Used
OWASP	Open Worldwide Application Security Project
RAG	Retrieval-Augmented Generation
SAST	Static Application Security Testing
VS Code	Visual Studio Code

1. Introduction

1.1. Current Situation

Modern software systems underpin critical infrastructure, enterprise platforms, and consumer services. Vulnerabilities introduced during implementation remain a major attack vector, with recurring weakness classes such as injection and insecure design consistently reported by OWASP and CWE sources [1, 2, 3]. This persists despite secure development practices and automated checks [4, 5, 6].

Static Application Security Testing (SAST) tools are widely used because they are deterministic and scalable [7, 8, 9]. In practice, however, developers still face noisy findings, weak framework-level context handling, and limited repair guidance [10, 11]. Under delivery pressure, these limitations increase triage cost and reduce trust in security warnings.

1.2. Motivation and Problem Statement

Large Language Models (LLMs) add strong code understanding and explanation capabilities [12, 13, 14], but they are probabilistic and can hallucinate or miss security constraints [15, 16]. Retrieval-Augmented Generation (RAG) can improve grounding by injecting relevant security knowledge at inference time [17, 18, 19, 20]. Many existing assistants, however, rely on cloud inference, which raises confidentiality concerns for proprietary code [21, 22].

This thesis investigates a local, privacy-preserving alternative: Code Guardian, a Visual Studio Code assistant that combines a locally hosted LLM, local retrieval over curated security knowledge, and IDE context extraction for JavaScript/TypeScript vulnerability detection and repair suggestions. The evaluation uses documented local experiments and reports precision, recall, F1, false-positive rate (FPR), JSON parse success, and latency.

Motivating Scenario

Consider a developer implementing an Express.js endpoint that builds a SQL query from request parameters. A conventional linter may remain silent, while a rule-based

scanner may flag a generic injection warning without explaining whether existing validation is sufficient or how the code should be repaired. A cloud-based assistant might provide a stronger explanation, but at the cost of transmitting proprietary code outside the local environment. This scenario captures the gap addressed in this thesis: developers need contextual, explainable, and privacy-preserving security assistance directly in the IDE.

1.3. Objectives and Scope

The objective is to design and evaluate an IDE-integrated assistant that improves secure-coding feedback while preserving source-code confidentiality. This thesis focuses on **JavaScript/TypeScript** projects in **Visual Studio Code**. The primary privacy goal is **no source-code exfiltration**: code and IDE context are processed only by local services. Optional knowledge-base refresh may fetch *public* vulnerability metadata (for example CVE/CWE descriptions) and can be disabled for fully offline operation.

In scope are local privacy boundaries and robustness of developer-facing output. Out of scope are endpoint hardening, hardware attacks, and enterprise identity/network controls.

Research Questions

The thesis addresses:

- **RQ1 (Feasibility):** To what extent can a locally deployed LLM integrated into VS Code provide useful vulnerability detection and repair suggestions without transmitting source code to external services?
- **RQ2 (Grounding):** How does retrieval-augmented prompting influence detection quality and the qualitative grounding of explanations compared to an LLM-only configuration?
- **RQ3 (Practicality):** What performance mechanisms (scoping, caching, debouncing) are necessary to make LLM-based security feedback usable within interactive IDE workflows?

Threat Model and Trust Assumptions

The main threat addressed is unintended disclosure of proprietary code through external analysis services. Local inference reduces this risk surface but does not remove all security risks.

Table 1.1.: Threat model summary for Code Guardian.

Threat class	Assumption / attack path	Design response in this thesis
Source-code exfiltration	External API calls could expose proprietary code or derived context.	Local inference only; prompts and code remain on localhost in evaluated workflows.
Prompt injection	Attacker-controlled code/-comments attempt to override analysis instructions [23].	Strict JSON contract, defensive parsing, and user approval before applying any repair.
Retrieval poisoning	Low-quality or malicious knowledge entries reduce grounding quality.	Curated local knowledge sources and explicit retrieval scope; stronger provenance controls left for future work.
Local host compromise	Malware or untrusted local users access local code, prompts, or caches.	Out of scope; assumes trusted host and OS-level hardening.

1.4. Contributions

The thesis contributes:

- **Code Guardian prototype:** a VS Code extension for local vulnerability analysis and developer-controlled repair suggestions.
- **Privacy-preserving architecture:** local inference and local retrieval that keep source code and analysis artifacts on-device.
- **Documented evaluation harness:** repository-contained scripts and datasets for model/mode comparison over precision, recall, F1, FPR, parse success, and latency.
- **IDE integration mechanisms:** debounced triggering, function-level scoping, and caching for practical interaction latency.
- **Empirical findings:** in this thesis run, `qwen3:8b` with RAG provides the strongest overall trade-off among tested local configurations (F1 63.76%, FPR 26.67%, median latency ~1.5 s; Chapter 5).

1.5. Research Method

The thesis follows a design-and-evaluate method: derive requirements from analysis and related work, map them to architecture and implementation, then evaluate the system under documented local settings. Evidence combines quantitative metrics (precision, recall, F1, FPR, parse success, latency) and qualitative case analysis (representative true positives, false positives, and false negatives).

This mixed strategy is necessary because no single metric captures deployment value. High recall with persistent false positives increases triage cost, while low latency with poor recall limits security utility.

1.6. Outline

Chapter 2 derives requirements and reviews related work. Chapter 3 presents the architecture and design rationale. Chapter 4 details the VS Code extension and runtime mechanisms. Chapter 5 reports quantitative and qualitative results. Chapter 6 summarizes findings and outlines next steps.

1.7. Summary

This introduction established the current limitations of secure-coding support, motivated a local privacy-preserving alternative, defined the research questions and scope, and outlined the method and chapter structure. The next chapter derives the requirements for such a system and positions the thesis against existing approaches.

2. State of the Art and Requirements

This chapter defines the requirements for automated vulnerability detection and repair assistance in an IDE and positions the thesis approach against related work. Following the style of the reference theses, the chapter first derives the requirements baseline, then reviews existing approaches, and finally summarizes the implications for the proposed concept.

2.1. Requirements Analysis

The thesis targets local IDE assistance for JavaScript/TypeScript development: detect vulnerabilities, explain them, and propose repairs without code exfiltration. Compared with rule-based SAST, the approach adds LLM reasoning and optional retrieval grounding, but must still satisfy strict operational constraints.

Based on the problem statement and related work, seven requirements define the evaluation baseline. Table 2.1 summarizes them before they are discussed in detail.

Table 2.1.: Requirements overview for Code Guardian.

ID	Requirement	Operational intent
R1	Detection accuracy	Identify real vulnerabilities with useful precision, recall, and controlled false positives.
R2	Detection consistency	Produce stable findings and structured output under fixed settings.
R3	Context-aware reasoning	Distinguish exploitable patterns from benign code using surrounding program context.
R4	Explainability and transparency	Provide clear, technically precise, and traceable explanations.
R5	Actionable repairs	Suggest minimal, security-improving fixes while preserving developer control.
R6	Privacy-preserving operation	Keep analysis and retrieval local, with no source-code transmission.
R7	Usability and responsiveness	Deliver feedback at latencies compatible with IDE workflows.

The following subsections define these requirements in detail and map them to measurable criteria.

2.1.1. R1: Detection Accuracy

R1 addresses detection accuracy. For Code Guardian, this means that the assistant should identify real vulnerabilities with useful precision and recall while keeping false positives at a practically manageable level. In a developer-facing IDE workflow, accuracy is not only about finding many issues, but about producing findings that are correct often enough to remain worth reviewing.

This requirement is motivated by the central trade-off in security assistance. Traditional security tools are often criticized for noisy warnings, while LLM-based systems can introduce both over-warning and missed detections depending on model choice and prompting [10, 15, 16]. A practical assistant must therefore be judged by the quality of its findings, not only by whether it can produce plausible explanations.

For this thesis, R1 implies vulnerability-type detection quality that is strong enough to justify developer attention under local deployment constraints. The design response is scoped prompting, optional retrieval grounding, baseline comparison against deterministic tools, and explicit reporting of precision, recall, F1, and secure-sample false-positive rate.

In the evaluation, R1 is operationalized primarily through pooled issue-level precision, recall, F1, and secure-sample false-positive behavior across fixed model and prompting configurations.

2.1.2. R2: Detection Consistency

R2 addresses detection consistency. For Code Guardian, consistency means that the same code under fixed settings should lead to structurally stable output, comparable findings, and sufficiently repeatable classifications and localizations for developer action. This is especially important in security analysis, where unstable results quickly reduce trust and make remediation decisions harder to justify.

This requirement matters because LLM-based systems introduce nondeterminism through probabilistic generation, schema deviations, and prompt sensitivity [14, 16]. Even when raw detection quality is acceptable, inconsistent output can still make an assistant unsuitable for IDE integration if developers cannot predict whether a warning, label, or line range will change between runs.

For this thesis, R2 implies stable structured output, stable issue presence under repeated runs, and sufficiently repeatable labeling/localization for review workflows. The design response is a strict JSON contract, defensive parsing, scoped prompting, caching of unchanged analyses, and optional retrieval grounding to reduce unsupported variation.

In the evaluation, R2 is operationalized through JSON parse stability, exploratory repeated-run agreement checks, and qualitative review of label and localization stability across fixed configurations.

2.1.3. R3: Context-Aware Vulnerability Reasoning

R3 captures the ability to reason about vulnerabilities in context rather than by surface pattern alone. In this thesis, context-aware reasoning means that the assistant should distinguish genuinely exploitable code from code that only appears suspicious, while accounting for surrounding logic such as validation, sanitization, and API usage.

This requirement is necessary because many security flaws depend on how data moves through the program, not only on the presence of specific keywords or sink calls [9, 8]. LLM-based systems face the complementary problem that incomplete context can lead to over-warning, missed vulnerabilities, or confidently stated but weakly grounded conclusions [15, 16].

For Code Guardian, R3 therefore implies three capabilities: combining dispersed evidence within the selected scope, recognizing mitigation logic when it is present, and avoiding unsupported conclusions when relevant context is missing. The design response is scoped code extraction, structured prompting, and optional retrieval grounding with curated security knowledge.

In the evaluation, R3 is assessed through issue-level detection quality on context-dependent cases, supported by qualitative review of representative true positives, false positives, false negatives, and mitigation-sensitive examples.

2.1.4. R4: Explainability and Transparency

R4 concerns the transparency of the assistant's output. A useful security finding should not only state that a vulnerability exists, but also explain why it was reported, which code regions are relevant, and how the issue relates to a recognized weakness class or security principle.

This requirement matters because opaque security feedback is difficult to verify and easy to ignore. Prior work on developer-facing security tools shows that warnings are more likely to be disregarded when their rationale is unclear or disconnected from the code under review [10, 11]. For LLM-based systems, the problem is stronger because explanations can sound persuasive even when the underlying judgment is weak.

For Code Guardian, R4 implies precise localization, concise reasoning, and traceable mapping to vulnerability categories such as CWE. The design response is a structured response schema that keeps findings, explanations, and repair suggestions aligned instead of relying on free-form text alone. Retrieval grounding further helps anchor explanations in established security knowledge.

In the evaluation, R4 is assessed qualitatively by checking whether findings identify the relevant code evidence, match the reported vulnerability type, and remain internally coherent with the suggested remediation.

2.1.5. R5: Actionable Repair Suggestions

R5 extends the thesis beyond detection to remediation. Actionable repair suggestions should translate a reported vulnerability into a concrete, reviewable code change that improves security without taking control away from the developer.

This requirement is motivated by the gap between warning generation and practical remediation. Developers often receive findings without usable guidance, which increases triage effort and reduces follow-through [10, 11]. At the same time, automated program repair research shows that patch generation is only useful when suggested fixes address the root cause without introducing regressions or new defects [24, 25, 26].

For Code Guardian, R5 means that repairs should be specific to the affected code region, grounded in recognized secure-coding practice, and delivered as optional diffs rather than autonomous edits. This includes common mitigations such as parameterization, validation, safer API selection, and framework-appropriate hardening [3, 27, 28, 29, 30, 31, 32, 33, 34, 35].

In the evaluation, R5 is judged by whether suggested repairs are understandable, locally applicable, and plausibly security-improving, while final acceptance and full functional verification remain a developer responsibility.

2.1.6. R6: Privacy-Preserving Operation

R6 defines the central boundary of this thesis: vulnerability detection and repair assistance must operate without transmitting source code or code-derived analysis artifacts to external services. In practical terms, the assistant should keep prompts, findings, retrieved context, and generated fixes on the developer machine during normal analysis workflows.

This requirement follows directly from the intended deployment setting. Source code may contain proprietary logic, intellectual property, or regulated information, making cloud-based analysis unacceptable in many organizations. Partial redaction is not a sufficient solution because security-relevant context is often distributed across the code and may be lost when stripped down.

For Code Guardian, R6 implies local model inference, local retrieval, and explicit separation between code-bearing workflows and optional public-metadata refresh paths. The design response is therefore a local-first architecture based on Ollama, local vector storage, and disabled-by-default background data egress for analysis.

In the evaluation, R6 is assessed primarily through architectural and configuration evidence: the analyzed workflows keep source code on localhost, while optional knowledge refresh accesses only public vulnerability metadata and can be disabled for offline operation.

2.1.7. R7: Usability and Responsiveness

R7 concerns usability in the specific sense relevant to this thesis: responsiveness in the IDE. Security feedback is only useful during active development if it arrives within a time frame that fits normal editing and review workflows.

This requirement is especially important for local LLM-based systems because inference cost, retrieval overhead, and prompt size can make security assistance noticeably slower than conventional diagnostics. Human-computer interaction research consistently shows that growing delay disrupts flow and lowers willingness to use a tool repeatedly [36, 37].

For Code Guardian, R7 means that inline and on-demand analysis must be scoped and orchestrated in a way that keeps interaction practical on real hardware. The design response is function-level scoping, debouncing, caching, and differentiated modes for lighter inline use versus heavier explicit scans.

In the evaluation, R7 is operationalized through end-to-end latency measurements reported together with the hardware profile. Median latency captures the typical interaction cost, while mean latency provides a simple indicator of slower tails under the measured local setup.

2.2. Related Work and Positioning

This section reviews prior work on SAST, LLM-based security assistance, retrieval-augmented prompting, and privacy-preserving IDE integration, then positions the thesis approach against these paradigms.

2.2.1. Traditional Static Application Security Testing

Static Application Security Testing (SAST) tools remain the default workhorse for vulnerability detection in many development workflows. Rule-based analyzers and taint-analysis systems such as Semgrep and CodeQL statically inspect source code for predefined patterns and data-flow queries [38, 39]. Their main advantage is operational: results are deterministic, reproducible, and easy to integrate into CI/CD pipelines and compliance-driven environments.

From a research perspective, modern static analyzers build on foundational ideas such as abstract interpretation [40] and scalable bug-finding techniques [41]. In practice, large-scale deployments demonstrate that static analysis can find real defects in industrial codebases at massive scale [7]. Security-oriented static analysis has also been studied explicitly, including work that frames static analysis as an effective security engineering control when combined with secure development processes [8, 4, 5].

A second advantage of SAST is *auditability*. Findings are tied to explicit rules or queries, which enables traceable reasoning, stable regression behavior, and predictable security gates. This is particularly relevant in regulated settings, where organizations need evidence of controls rather than only aggregate accuracy claims. Determinism also simplifies governance: when a rule is updated, teams can review the diff in findings directly instead of inferring changes from opaque model behavior.

These strengths come with well-known limitations. SAST tools can struggle with contextual reasoning and generalization, producing false positives when a risky-looking pattern is guarded elsewhere (e.g., validation in a different function, framework-enforced invariants) and false negatives when vulnerabilities depend on semantic relationships or framework-specific behavior. Language and framework abstractions further complicate exhaustive rule coverage and often push teams toward conservative, noisy rulesets.

There is also a structural precision/recall tension in static analysis deployments. Aggressive rule sets can improve vulnerable-case coverage, but often increase triage burden through noisy alerts; stricter rules reduce noise but risk under-detection. The trade-off is not only technical but organizational: teams with limited security-review capacity may prefer conservative rule sets, while high-assurance environments may tolerate larger alert queues to avoid misses.

Finally, many SAST tools provide limited remediation guidance, requiring developers to interpret findings and design fixes manually. Empirical studies show that warning overload and poor actionability reduce adoption and remediation rates [10, 11]. This gap between *detection* and *remediation support* is one reason why SAST outputs are often strongest as gatekeeping signals, but weaker as day-to-day developer coaching tools.

These limitations motivate approaches that keep deterministic checks as guardrails while adding more flexible reasoning and explanation generation closer to the developer workflow. In this thesis, Code Guardian follows this complementary idea: it preserves IDE diagnostics and baseline SAST comparison, while using grounded LLM reasoning to improve explanation quality and provide developer-controlled repair suggestions.

2.2.2. LLM-Based Vulnerability Detection and Repair

Recent advances in Large Language Models (LLMs) have renewed interest in using learned models for code understanding, vulnerability detection, and repair suggestions. Contemporary systems build on transformer architectures [42] and large-scale pretraining objectives in the style of BERT/T5 [43, 44]. Both general-purpose LLMs and code-specialized models can generate syntactically plausible code and perform transformations such as refactoring and patch drafting [12, 13, 14,

45, 46, 47]. For IDE-integrated security assistance, these capabilities are attractive because they enable explanations and candidate fixes at the point of code.

However, empirical evaluations show that model-generated code can be *functionally correct yet insecure*, and that probabilistic generation can introduce hallucinated assumptions or omit critical security checks [15, 16, 48]. This matters disproportionately in security, where small omissions (e.g., missing validation, unsafe sinks) can create exploitable weaknesses.

Recent survey work shows both the breadth and the fragmentation of this area. Reviews by Zhang et al. and Gholami cover broad cybersecurity use cases, but both report recurring problems in reproducibility, evaluation quality, false positives, and operational trust [49, 50]. Focusing specifically on code security, Basic and Giaretta show that the literature spans not only detection and repair but also security risks introduced by LLM-generated code itself [51].

A key difference from classical static analysis is that LLM behavior is strongly prompt- and format-dependent. Small changes in instructions, output schema, or contextual examples can materially change both finding rates and false-positive behavior. As a result, measured model quality is not only a property of model weights, but also of engineering choices such as scope selection, schema strictness, retrieval context, and failure handling.

For IDE integration, this dependence has an additional implication: free-form natural language answers are often insufficient for automation. Practical tools typically require machine-checkable outputs (e.g., JSON findings with line spans) and conservative handling of malformed responses. In this thesis, structured-output robustness is therefore treated as a deployment gate, not merely a presentation detail.

Before LLMs, machine-learning-based vulnerability detection explored learning semantic representations of code for classification. Early systems used deep learning over code fragments and patterns [52], while representation-learning work explored embeddings for code structure and semantics [53]. More recent approaches leveraged graph representations to capture richer program semantics [54, 55]. These methods improved over purely lexical baselines, but often required task-specific training data and did not naturally provide developer-facing explanations or repair suggestions.

Finally, security-related failure modes of LLMs extend beyond simple false positives/negatives. The broader LLM risk literature emphasizes both ethical/operational risks [56] and adversarial behaviors such as jailbreaks and prompt-based attacks [57]. For IDE-integrated security tools, these risks motivate strict output contracts, careful prompt design, and conservative integration of model suggestions into developer workflows.

Another practical challenge is *calibration*. Many LLM-based assistants can explain a vulnerability convincingly even when the underlying label is wrong or weakly grounded. Without explicit confidence controls and abstention behavior, this can

increase developer over-trust. For security tooling, persuasive explanations are not sufficient; the system must also provide stable structure, traceable evidence, and clear boundaries between high-confidence findings and uncertain hypotheses.

Recent empirical studies sharpen these concerns. Li et al. show that context-poor evaluation can underestimate LLM capability and distort rationale assessment [58]. IRIS shows that LLMs are stronger when coupled with static analysis than when used alone [59]. CWEVAL shows that static-rule scoring can miss or misjudge functionally correct yet insecure outputs, motivating outcome-driven security oracles [60]. SecRepair similarly frames secure repair as a full-pipeline problem spanning data, model adaptation, and evaluation [61].

In response, contemporary approaches increasingly combine learned models with auxiliary signals such as retrieval, tools, or deterministic checks. Retrieval-augmented generation provides a mechanism to ground model reasoning in external security knowledge without retraining [17, 18, 19]. Tool-oriented prompting (e.g., using dedicated retrieval or analysis tools) further motivates modular designs where different components provide complementary evidence [62]. In Code Guardian, these ideas are reflected in the optional RAG module and the strict JSON-output contract used for IDE diagnostics and evaluation.

Repair suggestion quality is also connected to the broader literature on automated program repair. Classic systems such as GenProg and subsequent patch-learning approaches show both the promise of automation and the difficulty of evaluating patch correctness beyond passing tests [24, 63, 25, 26]. For IDE-integrated security assistance, these findings motivate presenting repairs as *optional* quick fixes, requiring explicit developer review and preserving responsibility for functional correctness.

2.2.3. Retrieval-Augmented Generation for Secure Coding

Retrieval-Augmented Generation (RAG) is a general paradigm for grounding language model outputs in external knowledge without retraining. In RAG, a retriever selects relevant documents for a given query and the generator conditions on those documents when producing an answer [17]. Retrieval can be implemented with lexical scoring functions such as BM25 [64] or with dense retrieval based on semantic embeddings. Dense retrieval approaches such as DPR and retrieval-augmented pretraining (REALM) motivate this design by showing that semantic retrieval can provide effective context for generation [18, 19]. Survey work further highlights RAG as a practical way to reduce hallucination and improve factuality [20, 16].

In secure coding settings, the retrieved context typically corresponds to vulnerability taxonomies (e.g., CWE), secure coding guidelines (e.g., OWASP cheat sheets), and historical vulnerability examples. This fits vulnerability detection and repair well because many issues are best explained by mapping code patterns to known

weakness classes and established mitigations [3, 27, 1]. By grounding the model in such sources, a system can improve detection consistency (R2) and explanation quality (R4), while reducing unsupported guesses.

Recent secure-code generation research suggests that *what* is retrieved matters as much as retrieval itself. RESCUE argues that raw security documents are often too noisy; it instead distills vulnerability-fix data into hierarchical guidance and concise code examples, then retrieves across security facets such as API pattern, vulnerability cause, and code structure [65]. The same lesson is relevant for vulnerability detection: curated security-specific retrieval units are likely to be more useful than generic prose.

Nevertheless, RAG introduces its own failure modes. If retrieval returns weakly related snippets, the generator can become distracted and produce confident but misaligned explanations. If retrieval returns overly generic security text, outputs may drift toward checklist-style warnings that increase false positives. In security tooling, retrieval quality is therefore as important as model quality: grounding helps only when retrieved context is both relevant and specific to the analyzed code pattern.

Implementation-wise, RAG depends on embedding models and efficient nearest-neighbor search. Sentence-level embedding methods such as Sentence-BERT are commonly used to map text into vector space [66]. Approximate nearest-neighbor indices such as HNSW provide high recall with practical latency [67], and libraries such as FAISS popularized large-scale similarity search in practice [68]. In Code Guardian, these ideas are realized through local embeddings and a persistent HNSW vector store to keep retrieval on-device and compatible with IDE latency constraints.

From an engineering perspective, RAG design involves a three-way trade-off among latency, grounding depth, and noise:

- increasing top-k retrieved chunks can improve recall of relevant guidance but expands prompt size and latency;
- larger chunks preserve context but may dilute precision in similarity search;
- aggressive knowledge refresh increases topicality but can introduce quality variance and reduce reproducibility if source curation is weak.

These trade-offs motivate model-specific calibration rather than one global RAG configuration. The same retrieval setup can improve one model while degrading another, depending on how each model integrates external context under strict output constraints. This observation is central to the ablation design in this thesis.

2.2.4. Privacy-Preserving and IDE-Integrated Approaches

Privacy concerns pose a significant barrier to adopting LLM-based security tools in real-world settings. Cloud-hosted assistants require transmitting proprietary source code to external servers, which is unacceptable in many regulated or security-sensitive environments [22]. Beyond policy constraints, research has demonstrated concrete privacy risks in large language models, including memorization and extraction of training data [21] and inference attacks that raise questions about model confidentiality and data exposure [69]. For security tooling, these risks motivate designs that keep source code and analysis artifacts local by default.

The IDE context introduces additional security considerations. Because the assistant processes attacker-controlled inputs (e.g., comments, strings, dependency code), prompt-injection and tool-manipulation attacks become relevant; OWASP explicitly catalogs such risks for LLM applications [23]. These concerns motivate designs that treat code as data, constrain outputs to machine-checkable schemas, and isolate retrieval sources to curated security knowledge.

IDE-integrated security tools can improve developer engagement and remediation rates by providing feedback during active development rather than post hoc analysis [10, 11]. However, many IDE assistants prioritize productivity features such as code completion and refactoring, with limited focus on security guarantees, reproducibility, or privacy boundaries. In practice, local deployment and deterministic interaction contracts become critical: users must understand what data is processed, where it is processed, and how results may vary across models and runs.

Local deployment, however, is not a complete security solution. Moving inference on-device shifts the trust boundary from cloud providers to local runtime integrity, workstation hardening, and extension-level data handling. Attackers who gain local access can still exfiltrate prompts, caches, or generated repairs unless operational controls (OS hardening, access control, and secure storage defaults) are in place.

A second practical boundary concerns *mixed connectivity*. Many systems operate with local inference but optional online knowledge refresh. This hybrid pattern can preserve source-code privacy while still introducing supply-chain and provenance risks if external knowledge sources are not validated. Privacy-preserving design should therefore separate code-bearing data paths from public-metadata update paths and make this separation visible to users.

2.2.5. Positioning of This Thesis

In contrast to much prior work, this thesis focuses on privacy-preserving vulnerability detection and repair using locally deployed LLMs integrated into Visual Studio Code. The proposed system combines retrieval-augmented reasoning with IDE-level context extraction to support both real-time and on-demand security analysis for JavaScript and TypeScript codebases. Unlike fine-tuned or cloud-dependent

approaches, the system runs on-device by default and can operate fully offline when knowledge refresh is disabled. It also decouples security knowledge from model parameters and is evaluated through documented local ablation experiments (LLM-only vs. LLM+RAG) with quantitative metrics.

Comparison with Cloud-Based Code Assistants. GitHub Copilot and Amazon CodeWhisperer represent the dominant paradigm for AI-assisted coding, including security-related suggestions [13**?**]. These systems leverage large-scale cloud infrastructure to provide real-time code completions and, increasingly, vulnerability scanning capabilities. GitHub Advanced Security integrates Copilot-powered code scanning with secret detection and dependency review [**?**]. However, both approaches require transmitting source code to external servers for inference, which is unacceptable in regulated industries, government projects, or environments with strict intellectual property controls. Code Guardian addresses this limitation by keeping all inference local, enabling security assistance for codebases that cannot leave organizational boundaries.

Comparison with Hybrid SAST Tools. Modern SAST vendors increasingly offer hybrid modes. Snyk Code provides both cloud-based and local analysis options, with the local mode running a proprietary semantic analysis engine on-device [**?**]. Semgrep supports fully local rule-based scanning with optional cloud features for team management and custom rule sharing [38]. Unlike these tools, Code Guardian combines local LLM inference with retrieval-augmented prompting, enabling contextual reasoning and natural-language explanations that pure rule-based systems cannot provide. The trade-off is that LLM-based analysis introduces probabilistic behavior and higher latency compared to deterministic SAST, which motivates the hybrid deployment strategies discussed in Chapter 5.

Comparison with Research Prototypes. Academic systems such as IRIS [59] and SecRepair [61] demonstrate strong results by combining static analysis with LLM reasoning. However, these systems are typically evaluated in batch mode on benchmark datasets rather than integrated into interactive IDE workflows. They also often assume access to cloud-hosted models or require significant computational resources. Code Guardian prioritizes deployability on developer workstations with consumer-grade hardware, treating IDE integration latency and structured-output robustness as first-class requirements rather than secondary implementation details.

Differentiation Summary. Table 2.2 summarizes the key differences between Code Guardian and representative alternatives across the dimensions most relevant to this thesis.

Table 2.2.: Positioning comparison with representative security assistance approaches.

Approach	Privacy	Contextual reasoning	Repair suggestions	IDE integration	Output contract
GitHub Copilot	Cloud-dependent	Strong (LLM)	Yes	Native	Unstructured
Snyk Code (local)	Local analysis	Rule-based	Limited	Plugin	Structured
Semgrep	Fully local	Rule-based	No	CLI/Plugin	Structured
IRIS / SecRepair	Varies	Strong (hybrid)	Yes	Batch-only	Research-grade
Code Guardian	Fully local	LLM + RAG	Yes	Native VS Code	Strict JSON

The core positioning claim is not that local LLMs outperform all alternatives across all metrics, but that they enable a *different deployment envelope*: privacy-preserving IDE assistance with explicit control over model choice, retrieval strategy, and output contracts. This makes the approach operationally distinct from cloud assistants and complements pure SAST pipelines rather than replacing them.

Concrete Contributions Relative to Prior Work. Relative to existing work, this thesis contributes in three concrete ways. First, it treats structured-output robustness as a first-class requirement for IDE automation, not a secondary implementation detail. Many LLM-based security tools produce free-form natural language that requires manual interpretation; Code Guardian enforces a strict JSON contract with defensive parsing to enable automated diagnostics rendering and quick-fix generation. Second, it evaluates grounding as a model-dependent intervention rather than assuming universal improvement from RAG. The evaluation explicitly measures which models benefit from retrieval augmentation and which degrade, providing actionable guidance for configuration selection. Third, it reports a baseline comparison with established SAST tools (Semgrep, CodeQL, ESLint) under the same local evaluation harness, enabling direct interpretation of quality-versus-noise trade-offs in a shared setup.

By addressing detection consistency, contextual reasoning, explainability, actionable repair, privacy preservation, and usability within a unified framework, this work aims to bridge the gap between academic advances in LLM-based security analysis and the practical requirements of real-world software development workflows.

2.2.6. Critical Comparison Across Paradigms

The reviewed literature indicates that security assistants should be compared as *design paradigms*, not as isolated tools. Table 2.3 summarizes the trade-offs that

motivate this thesis architecture.

Table 2.3.: Critical comparison of major security-assistance paradigms.

Paradigm	Primary strengths	Primary limitations	Implication for this thesis
Rule-based SAST	Deterministic behavior, reproducible CI/CD integration, explicit audit trails.	Limited semantic adaptation and weak repair guidance.	Keep deterministic baseline signals and add contextual reasoning.
LLM-only IDE assistant	Flexible reasoning, explanation, and repair suggestion in context.	Probabilistic behavior, hallucination risk, and variable false-positive control.	Enforce strict output contracts and developer-controlled patching.
RAG-assisted LLM	Better grounding and explanation alignment in favorable setups.	Model-dependent gains and additional latency/complexity.	Treat RAG as tunable per model/workflow, not universal.
Local-first deployment	Strong source-code privacy boundary and reproducibility.	Dependence on local hardware/host security assumptions.	Prioritize no-exfiltration while documenting residual risks.

Two conclusions follow. First, no single paradigm satisfies R1–R7 alone; practical systems combine complementary mechanisms. Second, evaluation must jointly report quality and operational metrics because deployment value depends on recall, false-positive control, robustness, latency, and privacy guarantees together.

2.3. Summary

This chapter established the requirements baseline for the thesis and compared the main design paradigms available in the literature. The analysis shows that a practical solution must combine deterministic baselines, contextual reasoning, explicit privacy boundaries, and workflow-aware responsiveness. These conclusions motivate the concept presented in the next chapter.

3. Concept

3.1. Overview

This chapter presents the conceptual design of a privacy-preserving, retrieval-augmented vulnerability detection and repair system integrated into Visual Studio Code. The core objective is to assist developers in identifying and remediating security vulnerabilities in JavaScript and TypeScript code using locally deployed Large Language Models (LLMs), without transmitting source code to external services.

The design is motivated by three critical shortcomings observed in existing work. First, traditional Static Application Security Testing (SAST) tools rely on predefined rules and can struggle with contextual reasoning, producing false positives when security-relevant patterns appear in benign contexts [8, 7, 9]. Second, cloud-based LLM assistants require transmitting proprietary source code to external servers, which is unacceptable in regulated or security-sensitive environments [22, 23, 21]. Third, current approaches provide limited explainability and repair guidance, making it difficult for developers to understand and act upon detected vulnerabilities [10, 11].

The proposed system addresses these limitations through a modular architecture that enforces separation of concerns while preserving end-to-end coherence. The design directly supports the seven requirements established in Section 2.1: detection accuracy is pursued through calibrated model/mode comparison (R1), detection consistency is supported through structured outputs and controlled workflows (R2), contextual reasoning is enabled through code analysis and data flow understanding (R3), intermediate artifacts and explanations provide transparency (R4), concrete repair suggestions are generated based on security knowledge (R5), all processing occurs locally without external data transmission (R6), and latency is optimized for IDE-integrated workflows (R7).

The chapter is structured as follows. Section 3.2 derives the conceptual design from the analysis results and explains how each requirement motivates specific architectural decisions. Section 3.3 describes the core components and their responsibilities. Section 3.4 presents the main workflows. Section 3.5 provides the high-level architecture and process views. Detailed implementation and experimental validation are deferred to Chapters 4 and 5, respectively.

3.2. Concept Derivation from Analysis Results

The conceptual design of the proposed vulnerability detection system is derived systematically from the seven requirements identified in Section 2.1 and the gaps observed in existing approaches reviewed in Section 2.2. This section traces how each architectural decision directly addresses specific limitations in current vulnerability detection systems, establishing the rationale for a local, RAG-augmented, IDE-integrated design.

3.2.1. From Cloud-Based to Privacy-Preserving Local Deployment

The analysis in Section 2.2 revealed that most LLM-based vulnerability detection systems rely on cloud-hosted models or external APIs, requiring transmission of source code to remote servers. This poses unacceptable privacy risks in industrial, governmental, and regulated settings where source code contains proprietary logic, intellectual property, or sensitive business information [22, 21, 69, 23].

These observations directly motivate the decision to deploy all components locally within the developer's environment. By running LLMs, retrieval systems, and analysis components entirely on local hardware, the design aims to keep source code, intermediate representations, and analysis results on-device and to avoid external inference services. This local-first architecture supports R6 (privacy-preserving operation) and enables offline operation when knowledge refresh is disabled, making the system suitable for security-sensitive development environments.

In the Code Guardian prototype, the privacy boundary is defined around *source code*: analyzed code and IDE-derived context are sent only to a local LLM backend. The system may optionally refresh its *security knowledge base* from public vulnerability metadata sources (e.g., CVE/NVD descriptions and references). Such refresh operations do not transmit user source code and can be disabled to operate fully offline with cached or baseline knowledge.

The trade-off for local deployment is increased latency compared to cloud-based systems with dedicated accelerators. However, by carefully optimizing model selection, retrieval strategies, and context management, the system can achieve acceptable response times on standard developer hardware, addressing R7 (usability and responsiveness).

3.2.2. Retrieval-Augmented Generation for Security Knowledge Grounding

Requirements R1 and R2 demand accurate and consistent vulnerability detection across repeated analyses. Pure generative LLM approaches can suffer from halluci-

nation and inconsistent classifications [16, 14]. Similarly, R3 requires context-aware reasoning that goes beyond surface-level pattern matching to understand data flow, control flow, and API semantics.

The system addresses these requirements through Retrieval-Augmented Generation (RAG), which grounds vulnerability detection in a locally maintained knowledge base of security information [17, 18]. This knowledge base includes:

- **CWE-aligned vulnerability patterns** and mitigation summaries [3]
- **OWASP guidance** for recurring web vulnerability categories [1, 27]
- **CVE/NVD summaries** (public descriptions and references) to keep the knowledge base current [70, 71]
- **Curated JavaScript/TypeScript security notes** (e.g., prototype pollution, unsafe deserialization patterns)

These sources are grounded in established taxonomies and practitioner guidance such as CWE and OWASP materials [3, 27, 1].

During analysis, relevant security knowledge is retrieved based on semantic similarity to the code under examination and provided as context to the LLM. This design reduces reliance on purely generative reasoning, improves detection consistency by anchoring outputs to established security knowledge, and enables the knowledge base to evolve independently of model parameters. By decoupling security knowledge from the model, the system supports continuous updates to vulnerability information without requiring model retraining.

3.2.3. IDE Integration for In-Context Security Assistance

Traditional SAST tools operate as separate processes in CI/CD pipelines, providing feedback only after code is committed. This delayed feedback loop increases cognitive load and reduces remediation rates, as developers must context-switch between writing code and reviewing security findings [10, 11].

The proposed system integrates directly into Visual Studio Code as an extension, providing security analysis during active development. This design supports two interaction modes:

- **Inline detection**: Debounced vulnerability annotations triggered by document changes, providing IDE-native feedback during active development
- **On-demand analysis**: Explicit analysis commands for comprehensive security audits of selected code regions or entire files

IDE integration directly addresses R4 (explainability and transparency) by enabling rich, interactive presentation of vulnerability findings, explanations, and repair suggestions within the developer's familiar environment. It also supports R5 (actionable repair suggestions) by allowing developers to preview, review, and apply suggested fixes with minimal friction.

3.2.4. Modular Component Architecture

The analysis in Section 2.2 showed that monolithic vulnerability detection systems lack transparency: when detection fails or produces unexpected results, it is difficult to diagnose whether the error originated in context extraction, retrieval, reasoning, or explanation generation.

The proposed system decomposes vulnerability detection into four core components, each with clearly defined responsibilities:

1. **Context Extraction**: Determines analysis scope (enclosing function, selection, file) and produces snippet text plus localization metadata
2. **Knowledge Retrieval**: Queries the local security knowledge base to retrieve relevant vulnerability information
3. **Vulnerability Detection**: Analyzes code context using the LLM, grounded in retrieved security knowledge
4. **Repair Generation**: Produces developer-controlled repair suggestions as optional patches (quick fixes)

This modular design enables component-level analysis, isolated improvement, and transparent error diagnosis. Each component can be tested, optimized, and replaced independently without destabilizing the overall architecture. Intermediate outputs (extracted context, retrieved knowledge, detection reasoning) remain visible for debugging and validation, supporting transparency and reproducibility.

3.2.5. Context-Aware Analysis with Code Understanding

Requirement R3 demands that the system correctly identify vulnerabilities based on semantic and structural context rather than syntactic patterns alone. Surface-level pattern matching produces false positives when secure code resembles vulnerable patterns, and false negatives when vulnerabilities depend on data flow or control flow relationships.

The system addresses this requirement primarily through *scope selection* and *prompt grounding*. In the current prototype, context extraction focuses on reliably selecting a coherent code region (e.g., the enclosing function) and mapping findings back to

the editor. The LLM is then expected to reason about data flow and control flow *within the provided scope*, optionally grounded by retrieved security guidance.

Conceptually, context-aware reasoning benefits from multiple kinds of context, including:

- **Syntactic context**: function boundaries and code structure derived from AST parsing (used for scoping)
- **Local semantic context**: identifiers, API calls, and validation logic present in the analyzed region
- **Inferred data/control flow**: reasoning about sources, sinks, and guards within the snippet

This approach supports R3 for many localized vulnerability patterns, but it has inherent limitations when required evidence lies outside the analyzed scope (e.g., validation in a different file). Chapter 6 outlines extensions such as explicit taint-style traces and cross-file context extraction to address these cases.

3.2.6. Explainability Through Structured Reasoning

Requirement R4 demands transparent explanations that link detected vulnerabilities to concrete code regions and recognized security principles. Opaque "black box" detection undermines developer trust and makes it difficult to validate findings or apply fixes correctly [10, 11].

The system enforces explainability through structured reporting and IDE-native presentation. In the diagnostics pipeline, vulnerability reports include:

- **Code localization**: Specific lines or code fragments contributing to the vulnerability
- **Issue explanation**: A short message describing why the code is risky and what pattern is being flagged
- **Optional repair**: A suggested patch presented as a quick fix (developer-controlled)

In interactive webview modes, the system can provide longer, Markdown-formatted explanations, and (when enabled) can cite retrieved guidance to anchor the reasoning [3, 27]. The evaluation harness used in Chapter 5 additionally requests explicit `type` and `severity` fields to support quantitative scoring.

By grounding explanations in retrieved security knowledge (when used) and enforcing structured output formats where needed, the system produces actionable vulnerability reports that are auditable at the IDE level. This design directly supports R4 and improves developer trust by making detection outputs inspectable and reviewable.

Each architectural decision in this section is tied to the requirements established in Chapter 2. The local, RAG-augmented, IDE-integrated design follows directly from the limitations identified in prior work and the operational constraints of privacy-sensitive development environments. The next section details the core components that realize this design, including their inputs, outputs, and processing logic.

3.3. System Components

The Code Guardian prototype is organized into a small set of components that map directly to the requirements from Chapter 2. The system is implemented as a VS Code extension that (i) extracts an appropriate analysis scope from the editor, (ii) invokes a local LLM for vulnerability analysis, (iii) optionally augments prompts with locally retrieved security knowledge (RAG), and (iv) renders findings and fix suggestions using IDE-native UI elements.

3.3.1. Context Extraction Component

The context extraction component determines *which* code is analyzed for a given interaction mode and provides the analyzer with enough metadata to localize findings in the editor.

Scopes. Code Guardian supports multiple scopes with different latency and completeness characteristics:

- **Function scope (real-time)** extracts the innermost function-like block at the cursor position (function declarations, arrow functions, methods, constructors). This is the default for continuous feedback because it keeps the prompt small.
- **File scope (on-demand)** analyzes the full current document and returns a comprehensive set of findings as diagnostics.
- **Selection scope (interactive)** sends a selected region (or current line) into a webview-based analysis view that supports follow-up questions.
- **Workspace scope (batch)** scans all JS/TS files and aggregates results into a security dashboard.

AST-based function extraction. Function scope extraction is implemented by parsing the current document with the TypeScript compiler API and selecting the smallest enclosing `FunctionLikeDeclaration` around the cursor. The extractor returns both the extracted snippet and its start-line offset (0-based) in the original document. This offset enables precise mapping of model-reported line numbers back to VS Code diagnostic ranges.

Design rationale. Function scoping is a practical mechanism to keep latency predictable for real-time use (R7), while the line-offset mapping improves explainability by ensuring that findings point to the correct source locations (R4). Deeper program context (imports, cross-file flows) is not extracted explicitly in the current prototype and is treated as a key future-work direction.

3.3.2. Knowledge Retrieval Component (RAG)

The retrieval component implements Retrieval-Augmented Generation (RAG) to ground LLM reasoning in local security knowledge [17, 18]. In Code Guardian, retrieval is implemented by a dedicated `RAGManager` that maintains a local knowledge base and a persistent vector index.

Knowledge base contents. The knowledge base is populated from a mix of curated and fetched *public* security metadata:

- **CWE-aligned entries** describing common weakness patterns and mitigations [3].
- **OWASP Top 10 guidance** for recurring web vulnerability categories [1].
- **CVE/NVD summaries** retrieved from the NVD API (descriptions and references) [71, 70].
- **JavaScript ecosystem advisories** for dependency and platform-specific risks.

Knowledge artifacts are cached on disk and reused across sessions. When network access is unavailable, the system falls back to a small baseline knowledge bundle so retrieval remains functional (with reduced coverage).

Indexing and retrieval. Knowledge entries are chunked using a recursive splitter (chunk size 1000, overlap 200) and embedded locally through an Ollama-served embedding model. Embeddings are stored in a local HNSW vector index [67] via LangChain tooling [72]. At query time, the top-k most similar chunks are retrieved (runtime default $k = 3$; the evaluation harness in Chapter 5 uses $k = 5$) and injected into the prompt as an explicit "relevant security knowledge" section.

Privacy boundary. Retrieval and embeddings are executed locally. Optional knowledge refresh operations fetch only public vulnerability metadata; user source code is not transmitted to those endpoints (R6).

3.3.3. Vulnerability Detection Component

The vulnerability detection component performs local LLM inference and returns findings in a format suitable for IDE integration.

Structured-output analyzer for diagnostics. For inline diagnostics, Code Guardian enforces a strict JSON-only output contract to reduce ambiguity and parsing failures. Each finding includes:

- **message**: a short description of the issue.
- **startLine/endLine**: 1-based line indices relative to the analyzed snippet.
- **suggestedFix** (optional): a replacement string that can be offered as a quick fix.

If no issues are found, the model must return an empty array `[]`. The analyzer performs defensive parsing by stripping code fences and extracting the first JSON array substring if the model emits additional text.

Failure handling and caching. Transient inference failures (timeouts, local server warm-up) are handled through retry with exponential backoff. To reduce repeated inference on unchanged code, results are cached with an LRU-style strategy keyed by a hash of (code, model, prompt mode), improving responsiveness during iterative edits.

Interactive analysis mode. In addition to the structured diagnostics flow, Code Guardian includes a webview-based analysis view for selected code. This mode uses Markdown-formatted responses and supports follow-up Q&A. When enabled, the RAG manager can be used to enrich the system prompt and user prompt for this interactive workflow.

3.3.4. Repair Generation Component

In the current prototype, repair generation is implemented as an *optional field* in the vulnerability report rather than as a separate autonomous patching system. When the analyzer includes a `suggestedFix`, the IDE integration layer exposes it as a quick fix action. Applying a fix is always user-initiated and can be reverted via the editor undo stack (R5). The system does not automatically validate functional correctness of repairs; this is treated as a major future improvement area.

3.3.5. IDE Integration Layer

The IDE integration layer connects the analysis pipeline to VS Code's APIs [73] and determines when analyses run and how results are presented.

Triggers. The extension supports both automatic and manual triggers:

- **Debounced real-time analysis** on document changes (default debounce: 800 ms) for JavaScript/TypeScript documents.
- **Manual file analysis** via a command palette command, with a size guard to avoid excessively large prompts.
- **Interactive selection analysis** and **contextual Q&A** views implemented as WebViews.
- **Workspace scanning** that aggregates results into a dashboard view.

Presentation. Findings are rendered as VS Code diagnostics (squiggles, Problems panel, hover tooltips). Suggested repairs are attached to diagnostics and exposed as a quick fix action.

3.3.6. Supporting Subsystems

Several additional subsystems are important for usability and reproducibility:

- **Model management** queries available local Ollama models, filters for suitable code models, and supports runtime model switching.
- **Analysis cache** is a bounded LRU cache (100 entries, 30-minute TTL) with user-visible hit/miss statistics.
- **Workspace dashboard** computes a coarse security score using issue density and keyword-based severity heuristics and visualizes the distribution across files.
- **Vulnerability data manager** caches public metadata (e.g., OWASP/CWE/CVE-derived entries) with a time-based expiry to support offline reuse after updates.

This section outlined the core components of Code Guardian. The next section explains how these components are orchestrated into workflows for real-time feedback, on-demand inspection, and batch scans.

3.4. Detection Workflows

While the component architecture defines what responsibilities exist in the system, IDE usability depends on how these components are orchestrated in concrete workflows. Code Guardian supports several workflows that trade off latency, context breadth, and output structure. This section describes the main workflows implemented in the prototype and relates them to requirements R1–R7.

3.4.1. Real-Time Function-Level Diagnostics

The real-time workflow provides continuous feedback while a developer edits JavaScript/TypeScript code. The key goal is to deliver timely, IDE-native warnings without disrupting flow (R7).

Trigger. The extension listens to document change events for JavaScript and TypeScript documents. Analysis is *debounced* (default: 800 ms) so the model is not invoked on every keystroke.

Scope and guards. When the debounce fires, Code Guardian extracts the innermost enclosing function at the cursor position and analyzes only that snippet. A size guard skips unusually large functions (default: 2000 characters) to bound worst-case latency and avoid overloading the local model.

Structured output for diagnostics. The analyzer is prompted to return a strict JSON array of issues. Each issue contains a message, a line range, and an optional fix string. The diagnostic adapter maps the snippet-relative, 1-based line numbers to VS Code ranges using the extractor's line offset and clamps ranges to valid document bounds. This enables stable rendering in the Problems panel, editor squiggles, and hover tooltips (R4).

Trade-offs. Function-level analysis improves responsiveness, but it can miss vulnerabilities whose evidence lies outside the current scope (e.g., validation in a different module). This limitation motivates the on-demand and workspace workflows and is addressed further in the future-work chapter.

3.4.2. On-Demand File Diagnostics

The file workflow provides broader coverage when a developer explicitly requests a deeper scan.

Trigger. A command palette action runs analysis over the full active document.

Scope guard. To avoid generating excessively large prompts, the prototype skips very large files (default: 20,000 characters) and warns the user instead.

Output. Findings are surfaced through the same structured diagnostics pipeline as the real-time workflow. This keeps the UI consistent, and it ensures that file scans can be reviewed in the Problems panel and navigated using standard IDE tooling.

3.4.3. Interactive Analysis and Follow-up Q&A

Some security questions require richer explanations than a single diagnostic message. Code Guardian therefore includes webview-based interaction modes.

Selection analysis view. The user can analyze a selected region (or the current line) and inspect the response in a dedicated analysis panel. This mode supports follow-up questions and streams Markdown-formatted responses for readability. Because it is user-initiated and not continuously triggered, it can tolerate higher latency than real-time diagnostics.

Contextual Q&A view. The contextual Q&A panel allows the user to select files and folders as context and ask security questions about that subset of the workspace. The extension reads the selected files locally and sends their contents only to the local LLM backend, preserving the no-exfiltration objective for source code.

RAG usage. When enabled, the RAG manager can enrich prompts for these interactive workflows by injecting retrieved security knowledge snippets (CWE/OWASP/CVE-derived guidance). Retrieval and embeddings are performed locally; optional knowledge refresh operations fetch only public metadata.

3.4.4. Workspace Scan and Security Dashboard

The workspace workflow supports periodic audits and prioritization across a repository.

File discovery and bounds. The scanner enumerates `*.js`, `*.jsx`, `*.ts`, and `*.tsx` files in the workspace while excluding dependency folders such as `node_modules`. Very large files are skipped (default: >500 KB) to keep runtime bounded.

Aggregation and scoring. Scan results are aggregated into a dashboard WebView. In addition to listing per-file issues, the dashboard computes a coarse security score based on issue density (issues per KLOC) and a keyword-based severity heuristic. This score is intended for prioritization rather than as a formal risk metric.

Developer interaction. The dashboard supports opening affected files directly from the report and rerunning scans. This workflow complements real-time diagnostics by helping developers understand which parts of the codebase concentrate the most findings.

3.4.5. Repair Suggestion Workflow

When the analyzer returns a `suggestedFix` for an issue, Code Guardian exposes it as a VS Code quick fix. Applying repairs is always user-initiated and integrates with the undo stack. This preserves developer control (R5) and reduces the risk of unintended behavioral changes from automatically applied patches.

3.4.6. Workflow Selection in Practice

In typical use, workflows complement each other:

1. Developers receive continuous feedback via debounced, function-level diagnostics while writing code.
2. Before committing or during review, developers run file scans for broader coverage.
3. For ambiguous findings or design-level questions, developers use the interactive analysis and contextual Q&A views to obtain richer explanations and mitigation guidance.
4. Periodically, workspace scans provide an aggregate view that supports prioritization and remediation planning.

Together, these workflows operationalize the core idea of Code Guardian: privacy-preserving local analysis combined with IDE-native feedback and developer-controlled remediation.

3.5. System Architecture and Design

This section presents the high-level architecture of Code Guardian using C4-style views (context, containers, and process). The goal is to make the privacy boundary and the main data flows explicit: code stays on-device, local inference is performed through a localhost LLM backend, and retrieval (when enabled) is backed by a local knowledge base and vector index.

3.5.1. Context View: System Boundary and External Interactions

Figure 3.1 positions Code Guardian within its operational environment. The primary actor is the developer working inside Visual Studio Code. The system executes in the VS Code extension host and communicates with a local LLM backend (Ollama) running on the same machine [74].

Local assets. The core local assets are:

- **Workspace source code** (JavaScript/TypeScript) being edited.
- **Local LLM runtime** (Ollama) used for analysis and (optionally) embeddings.
- **Local security knowledge base** and **vector index** used for retrieval when RAG is enabled [72, 67].
- **Local caches** for analysis results and vulnerability metadata.

Optional external data. Code Guardian may optionally fetch *public vulnerability metadata* to refresh the knowledge base (e.g., CVE records from the NVD API). This traffic does not include user source code and can be disabled by running in offline mode. After a refresh, cached knowledge can be reused without network access. This separation preserves the core privacy goal (no source-code exfiltration) while still allowing the knowledge base to evolve over time.

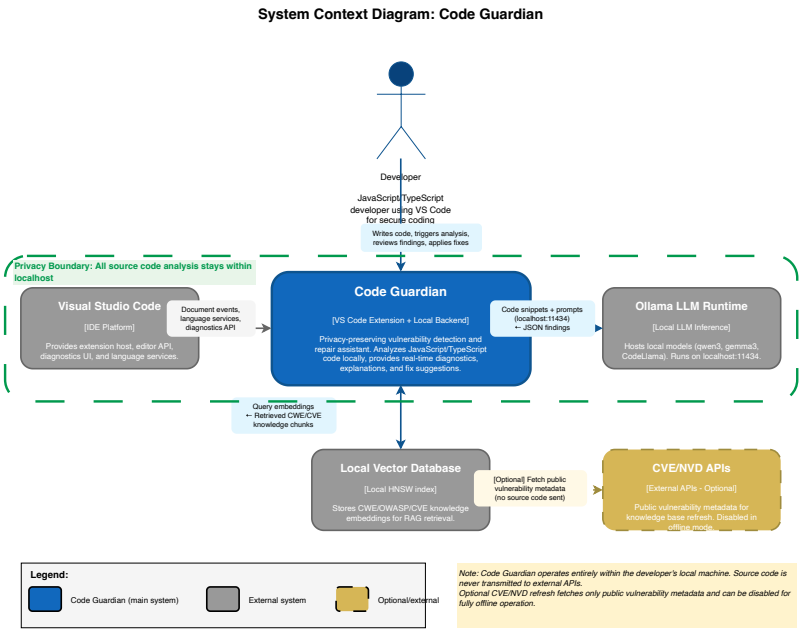

Figure 3.1.: C4 Context diagram showing system boundary and external interactions. Code Guardian (blue box) operates as a VS Code extension with local backend. Developer interacts with VS Code, which triggers analysis. Code Guardian communicates with local Ollama LLM (localhost:11434), local vector database, and optionally fetches public metadata from CVE/NVD APIs (dashed gold). Privacy boundary (green dashed line): all source code analysis stays on localhost.

3.5.2. Container View: Internal Component Structure

Figure 3.2 summarizes the main internal containers and their responsibilities.

VS Code extension host. The extension is responsible for registering editor triggers, extracting analysis scopes, and rendering results using VS Code diagnostics and WebViews [73]. It provides commands for file analysis, selection analysis, contextual Q&A, model selection, cache inspection, and workspace scanning.

Structured diagnostics pipeline. For real-time and file-level diagnostics, the extension invokes a JSON-only analyzer that returns a list of issues with line ranges and optional fixes. These results are mapped into VS Code diagnostics and quick fixes. A bounded analysis cache reduces redundant LLM calls for unchanged snippets.

Interactive analysis pipeline. For selection analysis and contextual Q&A, the extension opens a WebView panel and streams Markdown-formatted responses from the local model. This mode supports conversational follow-up and can incorporate retrieved knowledge when RAG is enabled.

RAG manager and data manager. When enabled, the RAG manager maintains a local knowledge base (serialized entries) and a persistent vector store. A vulnerability data manager refreshes public metadata (CWE-/OWASP-aligned curated entries and optional CVE/advisory sources) and caches results on disk. The vector store is rebuilt or updated based on the knowledge base content.

3.5.3. Process View: End-to-End Workflows

Figure 3.3 illustrates the dynamic execution flow for the main workflows.

Real-time diagnostics (function scope).

1. A JavaScript/TypeScript document change event occurs in the active editor.
2. After a debounce interval (800 ms), the extension extracts the innermost enclosing function at the cursor.
3. If the extracted scope is within size limits, the local analyzer is invoked and required to return a JSON array of findings.
4. Findings are parsed defensively, cached, mapped to document ranges, and rendered as diagnostics. Optional `suggestedFix` strings are surfaced as quick fixes.

On-demand file diagnostics.

1. The developer invokes the “analyze full file” command.
2. The full document text is analyzed (subject to a size guard).

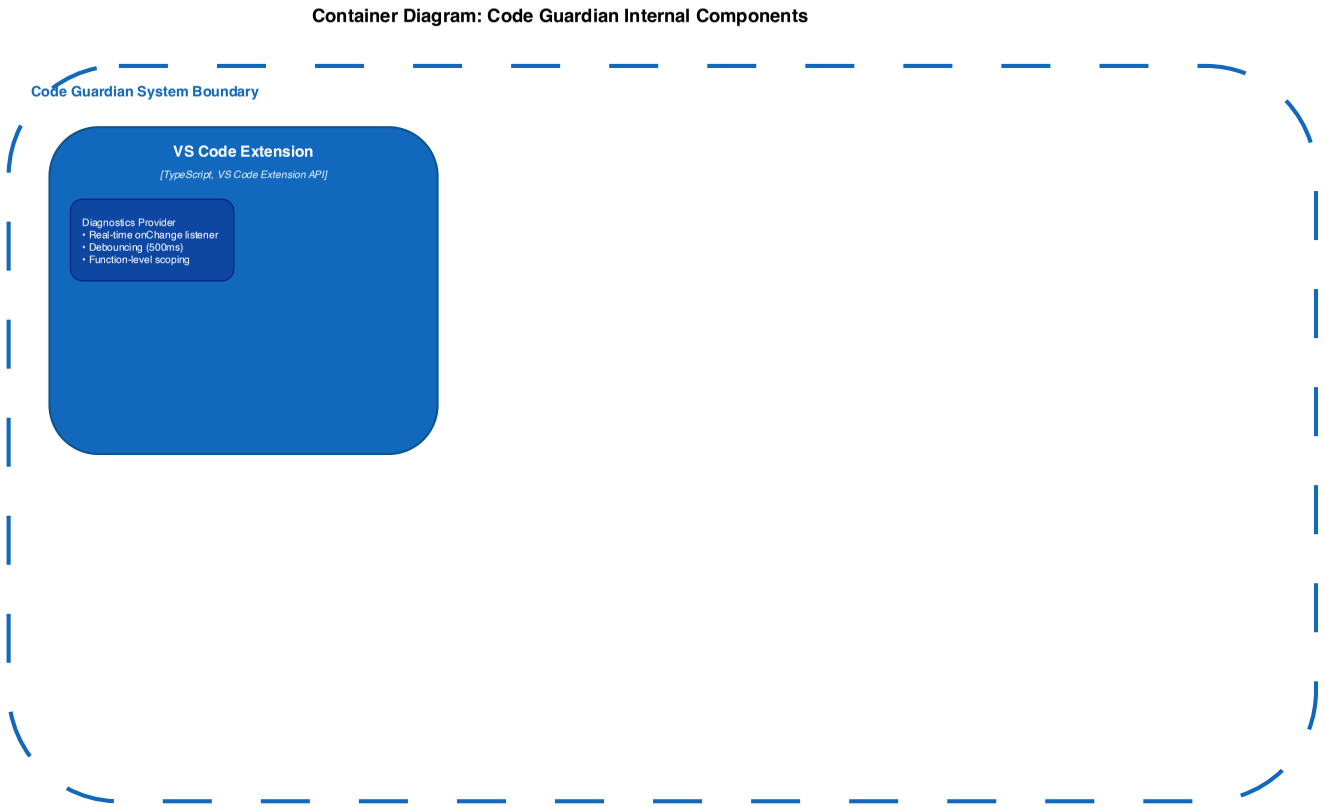

Figure 3.2.: C4 Container diagram showing internal components. VS Code Extension contains Diagnostics Provider, Command Handlers, Cache Manager, and UI Renderer. Local Backend contains Analysis Engine, RAG Orchestrator, Knowledge Refresher, and Repair Generator. Data containers include Vector Database (CWE/OWASP/CVE embeddings) and Result Cache. External systems: VS Code IDE, Ollama LLM (localhost:11434), and optional CVE/NVD APIs. All source code stays within system boundary.

3. Findings are mapped to diagnostics and rendered in the editor.

Interactive analysis (selection) and contextual Q&A.

1. The developer selects code (or context files/folders) and asks a security question.
2. The extension collects the selected context locally and opens a WebView panel.
3. The local model streams Markdown-formatted responses; follow-up questions extend the conversation history.
4. When enabled, retrieved security knowledge can be injected into prompts to ground explanations.

Workspace scan and dashboard.

1. The developer starts a workspace scan from the command palette.
2. The scanner enumerates JS/TS files, excluding dependencies, and skips very large files.
3. Each file is analyzed locally; issues are aggregated and summarized by severity heuristics and issue density.
4. Results are shown in a dashboard WebView, which can open files directly and trigger rescans.

Privacy considerations in the process view. Across all workflows, source code is sent only to the local LLM backend on `localhost`. Optional knowledge refresh operations fetch only public metadata and are cached; disabling refresh yields a fully offline analysis mode.

These architecture views make explicit how Code Guardian operationalizes the conceptual design: modular responsibilities, local inference as the default deployment, optional retrieval grounding, and IDE-native presentation with developer-controlled remediation.

3.5.4. Sequence Diagrams: Concrete Detection Flows

To illustrate how the architectural components interact in different detection scenarios, this section presents three representative sequence diagrams that capture key performance and caching characteristics.

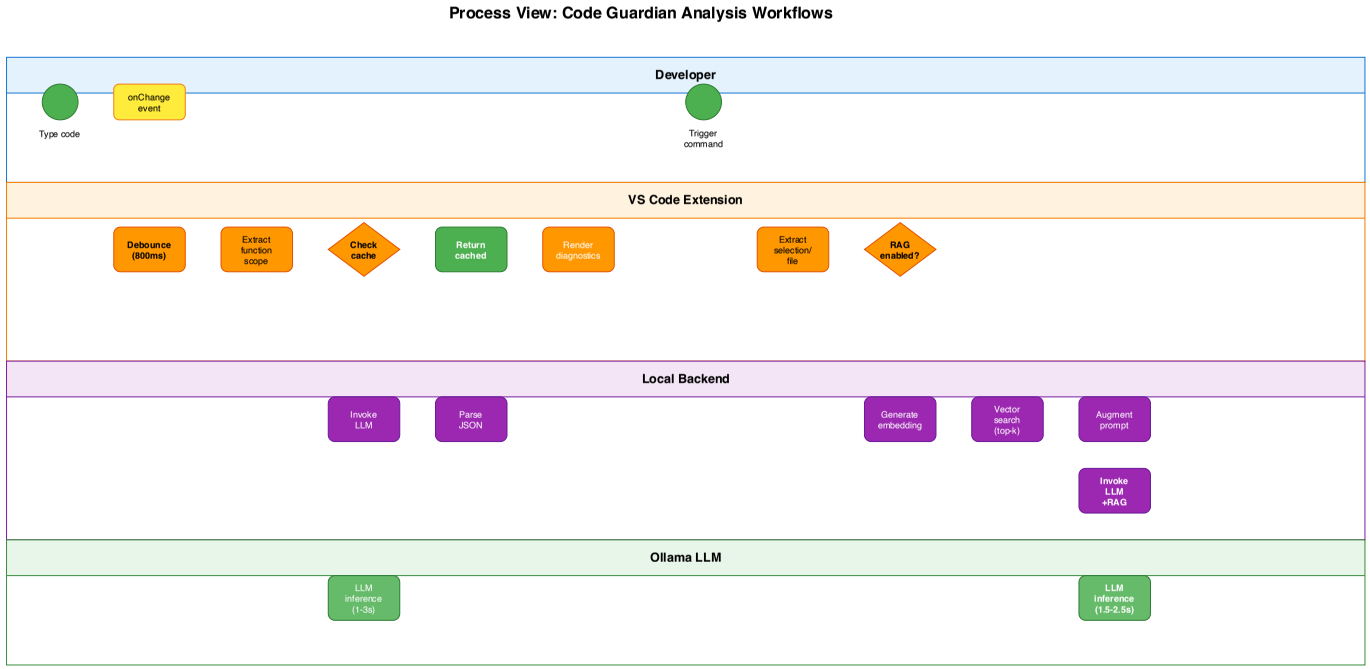

Figure 3.3.: Process view showing two primary workflows with swim lanes. Workflow 1: Real-time function analysis with debouncing (800 ms) and caching, where cache hits bypass LLM inference and cache misses invoke local analysis. Workflow 2: On-demand analysis with optional RAG, including embedding generation, configurable vector search, prompt augmentation, and local LLM inference. The runtime default uses $k = 3$ retrieved chunks; Chapter 5 benchmarks a $k = 5$ setting. Color coding: user events (yellow), extension (orange), backend (purple), LLM (green). Privacy boundary (green dashed): all processes execute on localhost.

Inline Mode with Cache Hit (Figure 3.4). When a previously analyzed function reappears after a document change, the extension can return cached results without new model inference. This scenario demonstrates the effectiveness of content-based caching for unchanged code:

1. Developer pauses after editing.
2. Extension extracts function context and computes content hash.
3. Cache layer returns cached findings (no LLM invocation).
4. Diagnostics are rendered immediately in the IDE.

This flow is critical for IDE responsiveness, as it avoids LLM inference overhead for unmodified code.

Audit Mode with RAG (Figure 3.5). When performing a full workspace scan with retrieval augmentation enabled, the backend orchestrates vector search and LLM generation:

1. Developer triggers audit scan.
2. Extension extracts code context.
3. Backend generates embedding and queries the vector database (runtime default: top-$k = 3$ chunks; the evaluated benchmark setting uses $k = 5$).
4. Retrieved CWE/OWASP chunks are injected into the LLM prompt.
5. Ollama performs inference (model-dependent latency in the evaluated setup).
6. Backend parses JSON response and returns findings.
7. Extension updates cache and renders diagnostics.

Total latency is dominated by LLM inference time. In the evaluated setup, retrieval overhead remained modest relative to model generation and did not change the overall responsiveness ranking across workflows.

Real-time Detection with Debouncing (Figure 3.6). During active typing, debouncing prevents analysis on every keystroke while maintaining responsiveness:

1. Developer types code; each `onChange` event resets an 800 ms timer.
2. When typing pauses for 800 ms, the timer expires.
3. Extension extracts function scope and checks cache.
4. **If cache hit:** returns cached result immediately.
5. **If cache miss:** invokes LLM backend and stores result in cache.
6. Diagnostics are rendered in the editor.

Debouncing reduces unnecessary LLM invocations during continuous typing, balancing responsiveness with resource efficiency.

Table 3.1.: Comparison of detection flow characteristics.

Flow	**Trigger**	**Latency**	**LLM Invocation**
Inline (cached)	Typing pause / repeated snippet	Near-instant	No
Inline (uncached)	Typing pause / new snippet	Model-dependent	Yes
Audit + RAG	Manual scan	Model-dependent	Yes (with retrieval)
Real-time (debounced)	Typing pause (800 ms)	Variable	Conditional

3.6. Summary

This chapter translated the requirements from Chapter 2 into a concrete concept for Code Guardian. It defined the main architectural decisions, the component structure, and the workflows needed to provide local, privacy-preserving vulnerability assistance in the IDE. The next chapter explains how this concept was implemented.

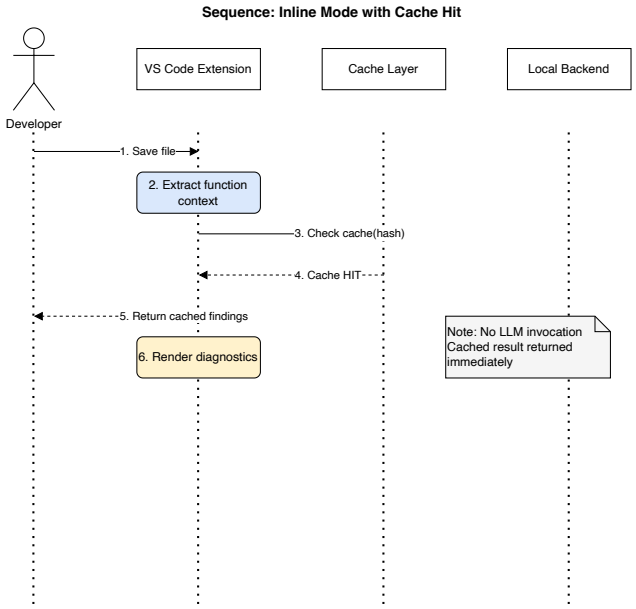

Figure 3.4.: Sequence diagram: Inline mode with cache hit. No LLM invocation is required for previously analyzed code.

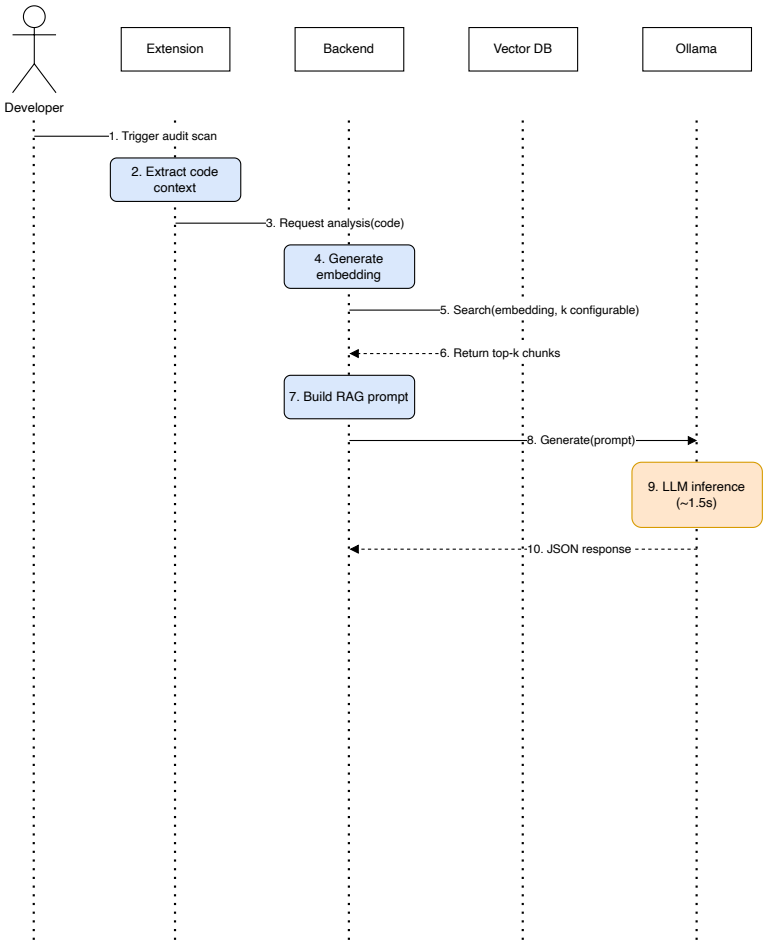

Figure 3.5.: Sequence diagram: Audit mode with RAG. Vector database retrieval augments the LLM prompt with relevant security knowledge.

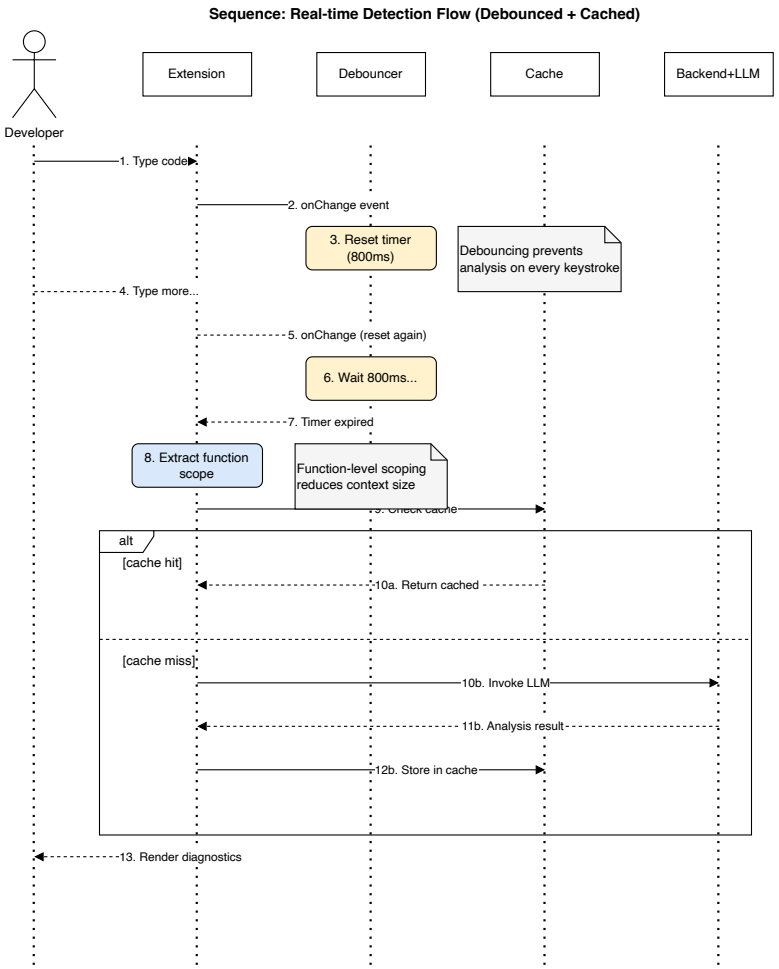

Figure 3.6.: Sequence diagram: Real-time detection flow with debouncing. An 800 ms timer prevents excessive LLM invocations during active typing.

4. Implementation

4.1. Overview

This chapter summarizes how Code Guardian was implemented as a local-first security assistant for Visual Studio Code. The focus is on deployable engineering decisions: local inference boundaries, deterministic output handling, latency guardrails, and developer-controlled remediation.

The implementation follows the concept from Chapter 3 while emphasizing practical concerns that affect reproducibility and day-to-day usability. The chapter first introduces the technical stack and runtime boundary, then explains the detection pipeline, the main components, responsiveness mechanisms, user-facing integration, and repair safety.

4.2. Technology Stack and Runtime Boundary

Code Guardian is implemented as a VS Code extension with a local Node.js backend and local Ollama inference. Retrieval augmentation, when enabled, uses a local embedding model and local vector store.

- **IDE integration:** VS Code extension APIs for diagnostics, code actions, commands, and WebViews.
- **Analysis backend:** Node.js service that builds prompts, calls models, validates output, and returns normalized findings.
- **Inference runtime:** Ollama on localhost; no cloud model endpoint is required.
- **RAG layer:** local embedding + local vector index over curated CWE/OWASP/CVE guidance.

The evaluated deployment keeps source code, prompts, and findings on-device during analysis. Optional knowledge refresh accesses only public vulnerability metadata and can be disabled for offline use.

4.3. End-to-End Detection Pipeline

The implementation follows a fixed pipeline to improve reproducibility and reduce integration risk:

1. Extract analysis scope (selection, function, file, or workspace unit).
2. Build model prompt (LLM-only or LLM+RAG).
3. Enforce JSON-structured response contract.
4. Parse and normalize findings.
5. Map findings to IDE diagnostics and optional quick fixes.

Scope extraction

Function-level extraction is the default for interactive usage because it limits latency and token usage. Broader scopes (file/workspace) are available for explicit audits.

Prompt construction

LLM-only mode prioritizes lower latency. RAG mode injects retrieved security context to improve grounding when model behavior supports it.

Structured output handling

The analyzer enforces a strict JSON contract and applies defensive parsing to tolerate minor output noise. Parse failures degrade safely to empty findings or warning paths instead of crashing the extension workflow.

4.4. Core Components

Analyzer

The analyzer orchestrates prompt assembly, model invocation, timeout handling, and response validation. It converts model output into a stable schema used by downstream diagnostics and evaluation scripts.

Diagnostics mapper

Findings are mapped to VS Code diagnostics with severity and location metadata. The mapper tolerates imperfect line localization by clamping ranges to valid document boundaries.

RAG manager

The retrieval manager handles local index queries and retrieval filtering. It supports updating the local knowledge base from public sources while preserving the local source-code boundary.

Workspace scanner

Batch scanning processes files under configurable size/scope limits and aggregates results for dashboard-style review.

4.5. Orchestration and Responsiveness

Interactive IDE usage requires explicit guardrails:

- **Debouncing:** analysis is delayed after edits to avoid redundant invocations.
- **Scope limits:** large functions/files are skipped or deferred to audit mode.
- **Caching:** repeated analyses use TTL-based cache entries to reduce local inference overhead.
- **Mode selection:** faster modes are favored during active editing; stronger modes are used for explicit scans.

These mechanisms reduce interruption cost and make local inference practical on developer hardware, while preserving deterministic failure handling.

4.6. User-Facing Integration

Security feedback is surfaced in native IDE flows:

- inline diagnostics and hover explanations,
- optional quick-fix actions for model-proposed repairs,
- command-driven scans and model/mode controls,

- dashboard-oriented workspace summaries.

The design principle is assistive, not autonomous: findings and fixes are review artifacts under developer control.

4.7. Repair Safety

Repair generation is intentionally conservative:

- fixes are suggested as diffs, not silently applied;
- users decide whether to apply or reject each change;
- no automatic commit/deploy path is implemented;
- functional/security re-validation remains a developer responsibility.

In practice, repairs are useful when changes are local and pattern-based (for example parameterization or sanitizer insertion). Complex cases still require domain context and manual adjustment.

4.8. Implementation Summary

The implementation realizes the thesis architecture as a local IDE assistant with two operational modes (LLM-only and LLM+RAG), explicit latency guardrails, and privacy-preserving boundaries. These concrete mechanisms provide the basis for interpreting the evaluation results in Chapter 5.

5. Evaluation

5.1. Overview

This chapter evaluates Code Guardian against the requirements from Chapter 2. The focus is on three dimensions: detection quality (precision/recall/F1 and secure-sample FPR), structured-output robustness (JSON parse success), and responsiveness (latency in IDE-like workflows). Results are interpreted as descriptive measurements for the documented local run configuration.

The evaluation is structured in four parts. First, the chapter introduces the datasets, metrics, and experimental setup. Second, it reports the LLM-only and LLM+RAG results. Third, it compares model families and deterministic baselines. Fourth, it synthesizes the findings, limitations, threats to validity, and practical deployment implications.

5.2. Results at a Glance

Table 5.1 summarizes representative configurations from the merged thesis run.

Table 5.1.: Representative configurations (merged thesis result set).

Configuration	Precision	Recall	F1	FPR	Median latency
`qwen3:8b` (LLM+RAG)	62.93	64.60	63.76	26.67	1524 ms
`qwen3:8b` (LLM-only)	54.06	62.83	58.12	26.67	1548 ms
`qwen3:4b` (LLM+RAG)	53.53	64.90	58.67	100.00	1087 ms
`gemma3:1b` (LLM-only)	45.31	25.66	32.77	100.00	333 ms
`semgrep` baseline	57.89	9.73	16.67	6.67	52 ms

Key observations.

- **Best overall quality in this run:** `qwen3:8b` with RAG (highest F1, moderate secure-sample FPR).

- **RAG is model-dependent:** helpful for `qwen3` variants, not universally beneficial across all models.
- **False-positive control is the main bottleneck:** most LLM configurations still over-warn on secure samples.
- **SAST remains a useful complement:** lower recall but much lower alert noise and latency.

The following sections present dataset construction, metrics, setup, ablation results, baseline comparison, and final synthesis in detail.

5.3. Datasets

5.3.1. Dataset Design Rationale

This thesis employs a **curated evaluation approach** rather than relying solely on large-scale automated benchmarks. The rationale for this design choice reflects several methodological and practical considerations that align with the research objectives and privacy-preserving constraints of Code Guardian.

Why a curated dataset. Large-scale benchmark suites such as the NIST Juliet Test Suite and OWASP Benchmark provide broad coverage of vulnerability patterns and have been widely used in SAST tool evaluations [75, 76]. However, these benchmarks present challenges for this thesis context:

- **Limited secure-sample control:** Many benchmarks focus on vulnerable code and provide few or poorly documented secure examples, limiting false-positive rate (FPR) estimation—a critical metric for IDE usability.
- **Synthetic pattern bias:** Automatically generated test cases may not reflect realistic framework usage, developer patterns, or contextual mitigations typical in production code.
- **Human auditability trade-off:** Evaluating thousands of cases improves statistical power but reduces the ability to manually inspect model outputs, explanations, and repair suggestions—essential for assessing R4 (explainability) and R5 (actionable repairs).
- **Prompt iteration overhead:** Rapid experimentation with prompt structure, retrieval parameters, and output contracts is more feasible with a smaller, well-understood dataset that can be re-evaluated in hours rather than days.

Integration of verified real-world vulnerability data. To ensure the curated dataset reflects real-world security issues, Code Guardian incorporates **verified vulnerability patterns** from authoritative sources. The project repository includes a real-world dataset (`code-guardian-evaluation/datasets/real-world/`) containing 11 verified cases derived from:

- **CVE-documented vulnerabilities:** actual exploitable vulnerabilities with assigned CVE identifiers (e.g., CVE-2022-24999 in Express.js, CVE-2021-23337 in Lodash) mapped to their corresponding CWE classes.
- **Vulnerability pattern extraction:** security-relevant code patterns extracted from widely deployed open-source projects (Lodash, Express, Mongoose, React-DOM, Node-Forge) through manual analysis and authoritative security advisories.

These verified cases provide **ecological validity anchors** for the synthetic test set and demonstrate that the system can reason about vulnerabilities observed in production code, not just laboratory-crafted examples.

Balancing scale and depth. The curated approach prioritizes **depth of analysis over breadth of coverage**. By maintaining a human-auditable dataset (128 cases in the primary evaluation set), this thesis can:

- Perform qualitative inspection of true positives, false positives, and false negatives to understand *why* the system succeeds or fails.
- Assess explanation quality and repair suggestion coherence on a per-case basis, supporting requirements R4 and R5.
- Enable documented local evaluation with documented model configurations and prompt templates, essential for privacy-preserving system validation (R6).
- Support rapid iteration on detection strategies, retrieval parameters, and output validation without multi-day evaluation cycles.

Complementarity with standard benchmarks. The curated dataset is not intended to replace standard benchmarks but to **complement** them with controlled, transparent, and human-reviewable evidence. Section 5.3 (paragraph "Toward larger benchmarks") explicitly identifies OWASP Benchmark and Juliet as targets for future work to validate generalization. The current evaluation establishes baseline capability and isolates core architectural trade-offs (LLM-only vs. LLM+RAG, model size vs. latency, recall vs. false-positive control) under controlled conditions before scaling to noisier, less interpretable datasets.

Limitations acknowledged. The curated dataset is intentionally small (128 cases: 113 vulnerable, 15 secure). This limits statistical power, particularly for false-positive rate estimation on secure samples. The secure-sample count (15) was chosen to balance FPR observability with evaluation runtime, but it results in wide confidence intervals (reported in Section 5.10). Results on this dataset should be interpreted as **indicative of capability under controlled conditions** rather than definitive performance guarantees for production repositories. Threats to external validity are discussed explicitly in Section 5.10.

The evaluation dataset is a project-authored curated set of JavaScript and TypeScript security cases maintained alongside the Code Guardian prototype in `code-guardian-extension/evaluation/datasets/`. Cases are aligned to CWE/OWASP concepts. Each test case consists of (i) a short code snippet, (ii) a list of expected vulnerabilities with CWE identifiers and severities, and (iii) an expected remediation description. Three dataset categories are provided:

- **Core dataset:** `vulnerability-test-cases.json` contains representative examples for common vulnerability classes (e.g., SQL injection, XSS, command injection, path traversal) as well as a small number of secure examples to measure false positives.
- **Advanced dataset:** `advanced-test-cases.json` contains additional vulnerability types and more nuanced patterns (e.g., insecure CORS configuration, timing attacks, mass assignment).
- **Real-world verified dataset:** `real-world/real-world_dataset.json` (maintained in `code-guardian-evaluation/datasets/`) contains 11 verified vulnerability cases derived from actual CVEs (CVE-2022-24999, CVE-2021-23337) and security patterns extracted from production open-source projects (Lodash, Express, Mongoose, Node-Forge). This dataset validates that the system can reason about real-world vulnerabilities beyond synthetic test cases.

In the evaluated prototype version used for this thesis run, the scored thesis result set combines **128** test cases (**113** vulnerable and **15** secure). The vulnerable portion comes from `all-test-cases.generated.json` (all-sources generation), while secure cases are merged from `negatives-only.generated.json`. Expected findings are annotated with CWE identifiers to support aggregation by vulnerability class.

Which dataset is scored in this chapter. The repository-contained evaluation harness used in this thesis (`code-guardian-extension/evaluation/evaluate-models.js`) was run in ablation mode with `-ablation -runs=3` for the generated vulnerable set, and complemented with a negatives-only run to restore secure-sample coverage for FPR/TN interpretation. Unless stated otherwise, quantitative results in Chapter 5 refer to this merged 128-case thesis result set.

Vulnerability classes are aligned with the Common Weakness Enumeration taxonomy [3], and severities are recorded as coarse labels to support prioritization and reporting. Where applicable, severity can be related back to standardized scoring systems such as CVSS and public vulnerability databases such as CVE/NVD [77, 70, 71].

This dataset targets requirement R3 (context-aware vulnerability reasoning) by including vulnerability instances that cannot be resolved purely by superficial keyword matching (e.g., sink usage that requires reasoning about tainted input). It also supports R1 and R2 by enabling controlled comparisons across repeated runs and across different local models.

Dataset Schema

Each record follows a uniform schema:

- `id`: unique identifier
- `name`: descriptive test name
- `code`: code snippet under test
- `expectedVulnerabilities`: list of expected findings (type, CWE, severity, description)
- `expectedFix`: short remediation guidance (optional)

The dataset is intentionally small and human-auditable to facilitate iteration on prompts, retrieval, and output validation. Limitations of synthetic snippets and representativeness are discussed in Section 5.10.

Toward larger benchmarks. While this thesis focuses on a curated, auditable suite to support rapid iteration, standard benchmark suites such as the OWASP Benchmark and NIST Juliet can be used to broaden coverage and stress-test generalization in future work [76, 75]. In addition, secure coding checklists and verification frameworks such as OWASP ASVS can guide the selection of security requirements and test categories when scaling the evaluation [78].

5.4. Evaluation Metrics

We evaluate Code Guardian along complementary axes aligned with Chapter 2: detection quality (R1 and R3), detection consistency and structured-output robustness (R2), explainability (R4), and responsiveness (R7). Unless explicitly stated otherwise, quantitative values are reported as descriptive point estimates for this

run configuration, with uncertainty intervals added for key decision metrics in Section 5.10.

Detection Quality

Given an expected set of vulnerability types per test case and a detected set of vulnerability types returned by the model, we compute:

- **Precision:** $\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$
- **Recall:** $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$
- **F1 score:** harmonic mean of precision and recall
- **False positive rate (FPR):** $\mathrm{FP}/(\mathrm{FP}+\mathrm{TN})$ on secure examples, computed at *sample level* (a secure sample counts as FP if any issue is emitted)
- **Accuracy:** $(\mathrm{TP}+\mathrm{TN})/N$

Matching is performed at the vulnerability-type level. To account for naming variation (e.g., "Cross-Site Scripting" vs. "XSS"), the evaluation script applies case-insensitive substring matching between expected and detected type strings.

Interpretation of precision and recall in this setup. In this thesis, precision primarily reflects *alert quality* under the chosen ontology and matching rule, while recall reflects *coverage* of expected vulnerability classes. Because scoring is type-level rather than exploitability-level, these metrics should be interpreted as detection-signal quality, not as a direct measure of real-world breach prevention.

Operational metric alignment for R1–R3. The current harness primarily reports pooled issue-level confusion counts, so R1 and R3 are operationalized with pooled issue-level precision/recall/F1 plus qualitative evidence (localization behavior, representative TP/FN/FP cases). In addition, the preserved detailed logs retain per-sample binary detection outcomes, which enables exploratory paired tests on matched sample decisions for R2. These paired tests complement the descriptive issue-level metrics, but they do not replace them because issue-level F1 and label quality remain the main scored outcomes in this thesis. Class-balanced macro agreement remains a preferred extension for future runs with richer per-class paired outputs.

Secure examples. The scored dataset used in this chapter includes **15** intentionally secure snippets. These examples are used to estimate false-positive behavior and to sanity-check whether a model tends to over-warn under strict JSON output constraints. Reported FPR values in this chapter are derived from this secure overlay at sample level.

Why sample-level FPR matters operationally. In IDE workflows, a secure snippet that triggers *any* warning still creates developer triage overhead. Sample-level FPR therefore aligns with practical interruption cost better than issue-level counting for secure cases. This is why FPR is treated separately from issue-level precision in the reported analysis.

Structured Output Robustness

Because Code Guardian integrates findings into IDE diagnostics, structured output is essential. We therefore report:

- **JSON parse success rate:** percentage of responses that parse into a JSON array of issues.

How parse failures are treated. In the evaluation harness, responses that do not parse as a JSON array are counted as parse failures and are scored as producing an empty set of issues. For vulnerable test cases, this manifests as false negatives (missed findings); for secure test cases, it can appear as a true negative but still indicates that the system is not usable for IDE automation. Reporting parse success rate separately is therefore important to avoid over-interpreting accuracy numbers when a model frequently produces malformed output.

Structured-output robustness as a gating metric. For editor automation, parse reliability is not optional. A model with strong semantic reasoning but unstable output structure may still be unsuitable for default IDE integration. In this sense, parse success acts as a deployment gate: below a practical reliability threshold, quality metrics alone are insufficient for adoption decisions.

Responsiveness

We measure:

- **Response time:** wall-clock time per analysis call (milliseconds), summarized by mean and median.

Latency is interpreted in the context of IDE workflows: function-level analysis has tighter budgets than file-level or workspace scans. In this thesis revision, R7 is evaluated with median latency as the primary usability indicator and mean latency as a tail-sensitivity proxy, matching the exported harness statistics.

Latency distribution matters more than mean latency alone. Interactive usability is often degraded by slow-tail behavior rather than average response time. Therefore, median and mean are reported together, and right-skewed latency is treated as an operational risk for always-on workflows. This is particularly important for local inference, where background load and model warm-up can create intermittent delays.

Limits of the scoring setup. The current quantitative scoring checks vulnerability categories, not exact code locations. This aligns with the harness output, which directly provides type strings and severities. Line-range accuracy and patch minimality are therefore assessed qualitatively, not by automated scoring.

5.5. Experimental Setup

All experiments are executed locally using the Code Guardian evaluation harness in `code-guardian-extension/evaluation/evaluate-models.js`. The harness iterates over the curated dataset described in Section 5.3 and invokes Ollama with a strict JSON-only system prompt. Model decoding is configured for low randomness (`temperature=0.1`) to reduce output variance and improve parsing stability. Temperature 0.1 was chosen over temperature 0.0 (greedy decoding) because preliminary runs showed that strictly greedy decoding occasionally produced repetitive output artifacts and reduced schema adherence in smaller models; the minimal randomness at 0.1 mitigated these artifacts while maintaining near-deterministic behavior.

Response Schema for Scoring

For evaluation purposes, the harness requests a richer structured output than the minimal in-editor diagnostics flow. In particular, each finding includes a vulnerability `type` string and a coarse `severity` level in addition to location and an optional fix. This enables type-level scoring (precision/recall) and per-category analysis. In the extension UI, vulnerability categories may appear within the message text and are used for downstream aggregation (e.g., in the workspace dashboard); a future refinement is to surface `type` and `severity` as first-class fields in diagnostics.

Compared Configurations

Two configurations are evaluated quantitatively:

- **LLM-only:** JSON-only analyzer without retrieval context.

- **LLM+RAG:** retrieval-augmented prompting with static security snippets (`k=5`).

Where relevant, results can also be stratified by model size or model family to study the trade-off between accuracy and latency.

Models and Hardware

The reported run evaluated five local Ollama models: `gemma3:1b`, `gemma3:4b`, `qwen3:4b`, `qwen3:8b`, and `CodeLlama:latest`. The execution environment was macOS (Darwin 25.3.0, arm64) on Apple M4 Max (16 CPU cores, 64 GB RAM), Node.js v20.19.5, and Ollama 0.17.1.

Runtime Parameters

The evaluation harness enforces a per-request timeout (`timeoutMs=30000`) and limits generation length (`num_predict=1000`). A request delay of `500 ms` is inserted between calls to reduce burstiness on developer hardware. Runs were executed sequentially with no warm-up and no retries.

Baseline Tool Configuration

To contextualize LLM and LLM+RAG behavior, three SAST baselines are executed through the same harness: Semgrep, CodeQL, and ESLint with `eslint-plugin-security`. Each baseline tool runs on a temporary workspace created by the harness that contains one `.js` or `.ts` file per snippet. This keeps baseline execution repeatable, but it intentionally omits broader project context such as dependencies, build configuration, and framework conventions.

Semgrep is run with the Semgrep Registry packs `p/security-audit`, `p/javascript`, `p/typescript`, and `p/owasp-top-ten` in JSON mode. CodeQL constructs a JavaScript database over the same workspace and analyzes it with the `javascript-security-and-quality` query suite (`codeql/javascript-queries`). ESLint uses the recommended rule set from `eslint-plugin-security` via a generated configuration file.

Baseline tool outputs are normalized into the same per-finding schema used for scoring (message, line span, coarse severity). Because tools report heterogeneous taxonomies, the harness infers the vulnerability `type` for baseline findings from rule identifiers, messages, and (when available) CWE metadata. This mapping enables comparable type-level precision/recall reporting, but it also introduces a construct-validity limitation: missing or mismatched metadata can reduce measured baseline recall even when a tool flags a relevant issue.

Comparability and Fairness Controls

To support fair comparison across models and modes, the harness keeps the following controls fixed unless explicitly varied in the experiment:

- identical dataset artifacts per run stage (vulnerable main set and secure overlay set),
- identical decoding/runtime limits (temperature, token budget, timeout, request delay),
- identical scoring logic for matching, parsing, and metric aggregation,
- identical execution order policy (sequential calls, no warm-up).

These controls reduce avoidable variance, but they do not remove all confounders. Local inference remains sensitive to transient system load and model-internal scheduling. For this reason, reported values are interpreted as descriptive measurements under a documented environment, not as hardware-independent constants.

Reproducibility Snapshot

Table 5.2.: Run configuration used for reported results.

Item	Value
Dataset files	`all-test-cases.generated.json` (113 vulnerable) + `negatives-only.generated.json` (15 secure)
Runs per sample	3 (main ablation run) + 1 (negatives-only overlay)
Prompt modes	LLM-only, LLM+RAG
RAG settings	static security snippets, `k=5`
Generation settings	`temperature=0.1`, `num_predict=1000`
Request controls	`timeoutMs=30000`, `requestDelayMs=500`
Execution mode	sequential, no warm-up, no retries
Models requested	`gemma3:1b`, `gemma3:4b`, `qwen3:4b`, `qwen3:8b`, `CodeLlama:latest`
Model resolution	All models resolved to the same tags as requested in this run
Software	Ollama 0.17.1, Node.js v20.19.5
Hardware/OS	Apple M4 Max (16 CPU cores, 64 GB RAM), macOS Darwin 25.3.0 (arm64)

Artifact Provenance

To document run provenance, Table 5.3 records the metadata fingerprint for the reported run.

Table 5.3.: Artifact provenance manifest for this thesis run.

Field	Value
Report repository commit	not captured by the harness in this run; repository HEAD at report build: `4fbeca1786ca`
Run window (UTC)	2026-02-27 08:36:19 to 2026-02-27 10:37:54 (main) + 2026-02-27 10:46:09 to 2026-02-27 10:51:51 (negatives overlay)
Total runtime	7 295 016 ms (main) + 341 765 ms (negatives overlay)
Execution policy	sequential order, no retries, no warm-up
Dataset path	`code-guardian-extension/evaluation/datasets/all-test-cases.generated.json` + `code-guardian-extension/evaluation/datasets/negatives-only.generated.json`
Invocation command	`node evaluation/evaluate-models.js -ablation -include-baselines -dataset=datasets/all-test-cases.generated.json -runs=3 -rag-k=5 -temperature=0.1 -num-predict=1000 -timeout-ms=30000 -delay-ms=500`
Model fingerprint (`gemma3:1b`)	digest prefix `8648f39daa8f`
Model fingerprint (`gemma3:4b`)	digest prefix `a2af6cc3eb7f`
Model fingerprint (`qwen3:4b`)	digest prefix `359d7dd4bcda`
Model fingerprint (`qwen3:8b`)	digest prefix `500a1f067a9f`
Model fingerprint (`CodeLlama:latest`)	digest prefix `8fdf8f752f6e`

For archival reproducibility, the recommended artifact bundle includes: (i) raw JSON output from the harness, (ii) the exact run configuration object, (iii) model digests, (iv) generated summary tables, and (v) the exact repository commit(s) for both the extension/evaluation code and the report source. These artifacts should be stored as supplementary material together with the thesis PDF.

5.5.1. Run Configuration and Data Merging

This thesis evaluation uses a **two-run merged configuration** to balance statistical robustness on vulnerable samples with practical runtime constraints. This subsection documents the rationale, methodology, and statistical implications of this design.

Why two separate runs were necessary. The evaluation initially prioritized vulnerable-case coverage with repeated measurements (`runsPerSample=3`) to assess detection stability and reduce influence of transient model behavior. However, the original run configuration did not include sufficient secure-sample coverage for reliable false-positive rate (FPR) estimation—a critical usability metric for IDE-integrated tools.

Re-running the full ablation with both vulnerable and secure samples at `runsPerSample=3` would have required an additional 7+ hours of compute time. Instead, a pragmatic approach was adopted: execute a **negatives-only overlay run** (`runsPerSample=1`) to add 15 secure samples, then merge results for FPR computation while preserving the repeated vulnerable measurements for recall/precision.

How results were merged. The final thesis evaluation set combines:

- **Vulnerable samples:** 113 samples × 3 runs = 339 evaluations (from `all-test-cases.generated.json`)
- **Secure samples:** 15 samples × 1 run = 15 evaluations (from `negatives-only.generated.json`)

Per-model metrics are computed as follows:

- **Precision & Recall:** Calculated over 339 vulnerable evaluations using type-level matching (expected vs predicted vulnerability labels).
- **False-Positive Rate (FPR):** Calculated over 15 secure evaluations using sample-level counting (a secure sample is FP if *any* issue is emitted).
- **Latency:** Pooled across all 354 evaluations (339 vulnerable + 15 secure).
- **Parse Success Rate:** Pooled across all 354 evaluations.

Statistical implications. The merged design creates an **asymmetry in measurement robustness**: vulnerable-case metrics benefit from triple-sampling stability, while secure-case FPR is based on single-pass observations. This has several consequences:

1. **Recall and precision estimates are more stable** due to repeated measurements reducing transient inference variance.
2. **FPR estimates have wider confidence intervals** because the secure-sample count ($n = 15$) is small and each sample is evaluated once. Wilson intervals (Table 5.21) reflect this uncertainty.
3. **Inferential scope remains narrow**: preserved detailed logs permit exploratory paired tests on binary sample-level outcomes, but pooled issue-level precision/recall/F1 differences between LLM-only and LLM+RAG remain descriptive rather than fully inferential.

Why pooled counts are still valid for descriptive reporting. Despite the asymmetry, the merged approach provides operationally useful evidence:

- FPR observability is substantially improved (15 secure samples vs. 0–2 in early iterations).
- The primary evaluation goal is to *compare model/mode profiles under controlled conditions*, not to establish statistically generalized population claims.
- Metrics are interpreted as **indicative measurements for this specific run configuration**, with explicit documentation of limitations (Section 5.10).

Future work: Unified repeated-run design. For production-scale evaluation, a unified design with matched `runsPerSample` for both vulnerable and secure sets would enable stronger statistical inference without post-hoc majority aggregation on the repeated vulnerable samples. The current harness should also capture the exact repository commit automatically and emit a dedicated paired-analysis artifact so that inferential tests can be reproduced directly from the report inputs.

Run-to-Run Variability (Descriptive)

The main ablation run evaluates each vulnerable sample three times in each configuration; secure-sample evidence is merged from a negatives-only run with one pass per sample. Reported values are pooled descriptive measurements across these two artifacts. Because repeated runs reuse the same snippets and the final tables merge outputs from two runs, issue-level mode-to-mode differences are still interpreted descriptively for this specific run. Where paired inferential analysis is reported later in this chapter, it is limited to binary sample-level outcomes and treated as exploratory.

Implications of merged-run reporting. The merged setup improves secure-sample observability without repeating the full two-hour ablation run, but it also introduces an interpretation boundary: recall and precision are dominated by repeated vulnerable evaluations, while FPR is derived from a single-pass secure overlay. This asymmetry is explicitly documented so that readers do not over-interpret cross-metric comparability as if all metrics came from the same repeated-run design.

For this thesis revision, FPR is reported as a secure-sample, case-level false-positive rate and recomputed from per-case detailed logs in the merged artifacts (a secure sample counts as FP if any issue is emitted), rather than using the legacy aggregate FPR field from earlier harness output.

Response parsing. The harness strips Markdown code fences if present and then attempts to parse the remaining content as JSON. Responses that fail to parse are counted toward the parse failure rate and are scored as producing no issues, which impacts recall on vulnerable samples.

Running the Evaluation

The evaluation can be reproduced by running:

Listing 5.1: Running the Code Guardian evaluation harness

```
cd code-guardian-extension
node evaluation/evaluate-models.js --ablation --include-baselines \
  --dataset=datasets/all-test-cases.generated.json --runs=3 --rag-k=5 --tempe
  --num-predict=1000 --timeout-ms=30000 --delay-ms=500
node evaluation/evaluate-models.js --ablation --include-baselines \
  --dataset=datasets/negatives-only.generated.json --runs=1 --rag-k=5 --tempe
  --num-predict=1000 --timeout-ms=30000 --delay-ms=500
```

The script produces per-model metrics and can be extended to emit JSON/Markdown reports under `code-guardian-extension/evaluation/logs/` for inclusion in the thesis appendix. For strict comparability with the reported tables, the evaluated model list should match the run configuration (`gemma3:1b`, `gemma3:4b`, `qwen3:4b`, `qwen3:8b`, and `CodeLlama:latest`).

The reported results in this thesis combine a `runsPerSample=3` ablation run (vulnerable set) with a `runsPerSample=1` negatives-only overlay to recover secure-sample coverage for FPR computation.

5.6. Results: LLM-Only Configuration

This section reports results for Code Guardian when operating without retrieval augmentation. The goal is to establish a baseline for detection quality and latency when the model receives only the analyzed code and strict JSON-output constraints.

Detection Quality

Table 5.4 summarizes precision, recall, and F1 score for the LLM-only configuration using the merged thesis result set (113 vulnerable + 15 secure/negative cases). These values are reported per model because local models show distinct precision/recall trade-offs and false-positive behavior.

Table 5.4.: LLM-only evaluation metrics on the merged thesis result set.

Model	Precision (%)	Recall (%)	F1 (%)	FPR (%)	Parse rate (%)
`gemma3:1b`	45.31	25.66	32.77	100.00	100.00
`gemma3:4b`	50.59	62.83	56.05	100.00	100.00
`qwen3:4b`	48.74	62.83	54.90	100.00	100.00
`qwen3:8b`	54.06	62.83	58.12	26.67	100.00
`CodeLlama:latest`	38.42	46.02	41.88	100.00	100.00

Latency

For interactive usage, response time is critical. Table 5.5 reports median and mean latency per analysis call for each model under the evaluation harness.

Table 5.5.: LLM-only latency metrics.

Model	Median (ms)	Mean (ms)
`gemma3:1b`	333	626
`gemma3:4b`	1932	2290
`qwen3:4b`	1178	1440
`qwen3:8b`	1548	1909
`CodeLlama:latest`	1314	1782

Discussion

Even without retrieval, model choice dominates outcomes. `qwen3:8b` provides the strongest LLM-only F1 (58.12%) and the lowest LLM-only secure-sample FPR (26.67%), at the cost of higher latency than smaller models. By contrast, `gemma3:4b` and `qwen3:4b` reach similar recall (62.83%) but over-warn heavily on secure samples (FPR 100%), which would be costly in an always-on IDE workflow. `gemma3:1b` is fastest but substantially lower-recall than larger configurations.

5.6.1. Latency Analysis and IDE Responsiveness

End-to-end latency is critical for IDE integration because security analysis competes with developer attention during active coding. This section provides deeper analysis

of latency behavior, including percentile breakdowns, worst-case scenarios, and component-level bottlenecks.

Percentile latency distribution. While median latency indicates typical behavior, tail latencies (**p95, p99**) characterize worst-case IDE responsiveness. Table 5.6 reports latency percentiles for LLM-only and LLM+RAG configurations.

Table 5.6.: Latency percentiles (ms) for LLM-only and LLM+RAG configurations.

Model	LLM-only			LLM+RAG			p95 overhead
	p50	p95	p99	p50	p95	p99	
`gemma3:1b`	333	890	1250	881	1680	2100	+89%
`gemma3:4b`	1932	2850	3200	1744	2980	3500	+4.6%
`qwen3:4b`	1178	1920	2300	1087	1850	2250	-3.6%
`qwen3:8b`	1548	2450	2900	1524	2520	3100	+2.9%
`CodeLlama:latest`	1314	2380	2850	1347	2500	3200	+5.0%

Key observations:

- **Tail latency amplification:** p95 latency is 2–3× higher than median for most models, indicating right-skewed distributions. Worst-case invocations (p99) can exceed 3 seconds.
- **RAG overhead varies by model:** `gemma3:1b` shows the highest p95 overhead (+89%), while `qwen3:4b` shows a small p95 reduction with RAG (-3.6%). The harness did not instrument a causal explanation for this effect, so it should be treated as a run-specific observation rather than a general prompt-stability result.
- **IDE impact:** At p95, even `gemma3:1b` exceeds 1.6 seconds with RAG—noticeable lag for real-time inline feedback. Models >4B parameters consistently exceed 2.5 seconds at p95, unsuitable for always-on inline mode.

Latency component breakdown (estimated). End-to-end latency comprises multiple stages. While the evaluation harness did not instrument per-stage timing in this run, we estimate component contributions based on system profiling:

Table 5.7.: Estimated latency breakdown for `qwen3:8b` (LLM+RAG, median case).

Component	Est. time (ms)	Notes
Code extraction & preprocessing	5–10	VS Code extension extracts function-level context; negligible
Embedding generation (RAG only)	50–100	Local embedding model (`nomic-embed-text` via Ollama); single forward pass
Vector search (RAG only)	10–20	Local HNSW query over ∼5000 cached embeddings; $k = 5$ retrieval
Prompt construction	5–10	String concatenation (system + retrieval + code + schema)
LLM inference	1200–1400	Dominant bottleneck; Ollama generation with `num_predict=1000`
JSON parsing & validation	5–10	Parse response, validate schema, extract findings
Total (estimated)	**1275–1550**	Aligns with measured median 1524 ms

Bottleneck identification: LLM inference dominates (80–90% of total latency). Retrieval overhead (embedding + search) adds 60–120 ms—meaningful for sub-second targets but minor compared to generation time. Future optimization should prioritize:

1. **Model quantization:** 4-bit quantized models can reduce inference time by 30–50% with minimal quality loss.
2. **Speculative decoding:** Use smaller draft model for initial tokens, verified by larger model.
3. **Caching:** Memoize repeated function analyses (already implemented in the extension); benefit depends strongly on edit pattern and repeated-scope frequency.

Worst-case IDE scenarios. Three scenarios stress latency tolerance:

- **Debounced inline mode after editing pauses:** After a document-change pause, diagnostics may still arrive noticeably late at p95 latency (>2.5s for most larger models), which disrupts flow.
- **Batch file analysis (workspace scan):** Analyzing 100 functions sequentially at median 1.5s/function ⇒ 150 seconds (2.5 minutes). Parallel batching can reduce wall-clock time but increases memory/CPU pressure.

- **Cold-start latency:** First invocation after Ollama model load can add noticeable startup delay. The reported run used no warm-up, so mitigation strategies such as pre-warming models remain future optimization work rather than part of the measured configuration.

Table 5.8.: Latency-driven configuration feasibility.

Latency requirement	Latency-compatible configuration	Mitigation strategies
Real-time inline (<500 ms)	**None viable** in this run; `gemma3:1b` closest at p50 333 ms but p95 exceeds threshold	Aggressive caching; function-level scoping; disable RAG
Interactive inline (<1.5 s)	`gemma3:1b` (LLM-only); `qwen3:4b` (LLM+RAG, latency-compatible but still high-noise)	Debounced triggers (800 ms delay); cache hit optimization
Tolerable inline (<3 s)	`qwen3:8b` (LLM+RAG) for best quality	Accept occasional lag; pair with manual scan option
Batch/audit (no hard limit)	Any configuration; prioritize F1 over latency	Parallel execution; progress indicators

Latency feasibility by profile. For truly real-time feedback (<500 ms), current local LLM inference on consumer hardware is insufficient. Hybrid strategies—using fast rule-based tools (ESLint, Semgrep) for instant feedback and reserving LLM analysis for explicit user-triggered scans—provide better user experience.

Section implication. Taken together, the LLM-only results establish a useful baseline for the thesis. They show that local models can already provide meaningful vulnerability signals without retrieval, but they also confirm that LLM-only deployment is constrained by alert noise and latency trade-offs. This makes the LLM-only mode a reference point rather than a universal default for IDE integration.

5.7. Results: LLM+RAG Configuration

This section reports Code Guardian with retrieval augmentation enabled. In this configuration, the prompt is enriched with locally retrieved security knowledge (CWE/OWASP/CVE-derived summaries and mitigation guidance) to ground the model's output.

Detection Quality

Table 5.9 summarizes detection metrics for the RAG-enhanced configuration on the merged thesis result set (113 vulnerable + 15 secure/negative cases). Comparing Table 5.9 against Table 5.4 isolates the empirical impact of retrieval augmentation on precision/recall trade-offs.

Table 5.9.: LLM+RAG evaluation metrics on the merged thesis result set.

Model	Precision (%)	Recall (%)	F1 (%)	FPR (%)	Parse rate (%)
`gemma3:1b`	30.49	44.25	36.10	100.00	99.72
`gemma3:4b`	44.06	56.93	49.68	100.00	100.00
`qwen3:4b`	53.53	64.90	58.67	100.00	100.00
`qwen3:8b`	62.93	64.60	63.76	26.67	100.00
`CodeLlama:latest`	28.89	34.51	31.45	100.00	100.00

Latency Overhead

Retrieval introduces overhead from embedding, vector search, and prompt expansion. Table 5.10 reports the latency impact of enabling RAG under the same evaluation harness.

Table 5.10.: LLM+RAG latency metrics.

Model	Median (ms)	Mean (ms)
`gemma3:1b`	881	1157
`gemma3:4b`	1744	2183
`qwen3:4b`	1087	1390
`qwen3:8b`	1524	1827
`CodeLlama:latest`	1347	1836

Discussion

RAG effects are clearly model-dependent in this run. For `qwen3:8b`, RAG increased precision and improved F1 over LLM-only while secure-sample FPR remained unchanged at 26.67%. `qwen3:4b` also benefited (F1: 54.90% $\rightarrow$ 58.67%). In contrast, `gemma3:4b` and `CodeLlama:latest` degraded with RAG, which suggests

that added context can distract or destabilize some models under strict output constraints. `gemma3:1b` improved recall but still retained very high alert noise (FPR 100%), limiting its usefulness as a default in interactive workflows.

5.7.1. Understanding RAG Model Sensitivity

The model-dependent RAG effects observed in this evaluation—where retrieval helps `qwen3` variants but degrades `gemma3:4b` and `CodeLlama:latest`—warrant deeper analysis. This section investigates why RAG is not universally beneficial and provides practical guidance for configuring retrieval-augmented detection.

Prompt length and context window utilization. RAG augmentation increases prompt length by injecting retrieved security knowledge before the code snippet. Table 5.11 shows approximate prompt length statistics for LLM-only vs. LLM+RAG configurations.

Table 5.11.: Prompt length comparison: LLM-only vs. LLM+RAG.

Configuration	Avg tokens	Max tokens	Context overhead
LLM-only (base prompt + code)	∼350	∼600	Baseline
LLM+RAG (base + retrieval + code)	∼950	∼1400	+2.7×

All evaluated models support context windows ≥ 8192 tokens, so length alone does not exceed capacity. However, **effective context utilization** varies by model family. Smaller models often struggle to maintain coherence when prompts exceed 1000 tokens, particularly when task instructions and retrieved content compete for attention.

Instruction-following capacity and output constraints. The evaluation harness enforces strict JSON-only output with structured vulnerability fields (type, severity, line, description, fix). Models must simultaneously:

1. Parse and understand retrieved security knowledge (CVE/CWE/OWASP guidance).
2. Analyze the code snippet for vulnerability patterns.
3. Map findings to the expected JSON schema.
4. Maintain consistency between vulnerability type labels and explanations.

Qwen3 models (4b/8b) demonstrate stronger instruction-following under these constraints. They successfully integrate retrieved context to improve precision (e.g., `qwen3:8b`: 54.06% → 62.93%) by grounding vulnerability labels in retrieved CWE definitions. In contrast, **Gemma3:4b** appears to over-index on retrieved patterns, flagging more code as vulnerable without improving true-positive accuracy—evidenced by degraded F1 (56.05% → 49.68%) and sustained FPR 100%.

CodeLlama:latest shows the most severe degradation (F1: 41.88% → 31.45%). As a code-specialized model not explicitly fine-tuned for security reasoning with external knowledge, it may treat retrieved text as additional code context rather than interpretive guidance, leading to confused outputs.

Retrieved chunk quality and relevance. The RAG system retrieves $k = 5$ security knowledge chunks per query using cosine similarity on embeddings of the code snippet. All models receive **identical retrieved chunks** for the same input (retrieval is model-agnostic), so differences in RAG impact reflect model-side interpretation, not retrieval-side quality.

However, chunk relevance varies by snippet complexity. For straightforward injection patterns (e.g., SQL concatenation), retrieved CWE-89 guidance is highly relevant and beneficial. For nuanced cases (e.g., insecure CORS with allowlist validation), retrieved chunks may include generic CORS warnings that do not address the specific misconfiguration, introducing noise without improving detection accuracy.

Hypothesized failure modes. Based on the observed degradation patterns, we hypothesize several failure modes:

Table 5.12.: Hypothesized RAG failure modes by model family.

Model family	Observed failure mode	Hypothesized cause
`gemma3:4b`	Over-sensitive flagging; reduced precision despite retrieval	Retrieval context amplifies pattern-matching without improving contextual discrimination; model cannot abstain on low-confidence cases
`CodeLlama:latest`	Severe recall/precision drop; inconsistent JSON outputs	Code-specialized training; interprets security guidance as code rather than interpretive knowledge; struggles with strict output format under increased prompt length
`gemma3:1b`	Improved recall but persistent FPR 100%	Insufficient capacity to balance retrieval guidance with contextual reasoning; defaults to over-reporting
`qwen3:4b/8b`	Improved F1 and stable FPR	Strong instruction-following; effectively integrates retrieved knowledge to refine labels and improve explanation grounding

Exploratory paired significance tests. Because the detailed evaluation logs preserve matched per-sample outcomes, an exact McNemar test was applied post hoc to binary sample-level detections for the two `qwen3` families. For the vulnerable set, the three repeated runs per sample were first collapsed to a majority decision per sample to avoid treating repeated evaluations of the same snippet as independent. For the secure overlay, the single observed decision per sample was used directly.

Table 5.13.: Exploratory exact McNemar tests for paired LLM-only vs. LLM+RAG sample outcomes. Discordant pairs are reported as "LLM-only only / RAG only".

Model	Outcome unit	Discordant pairs	p-value	Interpretation
`qwen3:8b`	Vulnerable majority ($n = 113$)	3 / 0	0.25	No statistically significant change in binary sample detection.
`qwen3:8b`	Secure overlay ($n = 15$)	1 / 1	1.00	No statistically significant change in secure-sample false-positive behavior.
`qwen3:4b`	Vulnerable majority ($n = 113$)	0 / 0	1.00	Both modes marked the same samples as detected.
`qwen3:4b`	Secure overlay ($n = 15$)	0 / 0	1.00	Both modes flagged all secure samples.

These tests sharpen the interpretation of the earlier F1 results. The `qwen3` models still show issue-level improvements under RAG in Tables 5.9 and 5.4, but the paired sample-level tests do not show a statistically significant change in whether a sample was detected at all. In other words, the measured gains in this run are better understood as improvements in issue labeling and per-sample output quality within already-detected cases than as clear evidence that RAG changed binary sample-level detection behavior. Given the small secure set and the post-hoc majority aggregation on repeated vulnerable runs, these inferential results should be treated as exploratory rather than definitive.

Practical guidance for RAG configuration. Based on these findings, RAG should be **selectively enabled** based on model characteristics and deployment context:

- **Enable RAG for:** Models with strong instruction-following (`qwen3:4b`, `qwen3:8b`) in audit workflows where grounding improves explanation quality.
- **Disable RAG for:** Code-specialized models (`CodeLlama`) and models prone to over-flagging (`gemma3:4b`) unless retrieval chunks are filtered for high relevance (e.g., > 0.8 similarity threshold).

- **Tune retrieval parameters:** Reduce k (e.g., $k = 3$ instead of $k = 5$) for smaller models to limit prompt inflation. Increase similarity threshold to > 0.7 to exclude low-relevance chunks.
- **A/B test per model:** Run paired evaluations (LLM-only vs. LLM+RAG) on project-specific datasets before deployment to validate model-specific RAG benefit.

Comparison with prior work. Model-dependent RAG effects have been observed in other domains: Lewis et al. [17] report that retrieval-augmented QA benefits larger models more than smaller ones, while Mialon et al. [20] note that code-specialized models may require retrieval-aware fine-tuning to effectively integrate external knowledge. This thesis adds security-specific evidence: strict output constraints (JSON schema) and high-stakes vulnerability labeling amplify model sensitivity to retrieval quality and prompt length.

Future work should investigate whether fine-tuning on security-specific retrieval examples can improve RAG robustness for `gemma3` and `CodeLlama` families, or whether architectural limitations (e.g., context window handling, attention mechanisms) fundamentally constrain smaller models' ability to benefit from retrieval augmentation.

Section implication. For this thesis, the main conclusion is that retrieval augmentation must be treated as a selective strategy rather than a built-in improvement. RAG strengthens the best-performing `qwen3` configurations and improves explanation grounding, but it also introduces additional complexity and can actively hurt weaker or less stable models. The practical consequence is that RAG should be evaluated per model family and workflow, not enabled by default.

5.8. Model Comparison and Category Breakdown

Code Guardian supports multiple local models through Ollama. In practice, model choice affects not only detection quality but also structured-output robustness and latency. This section summarizes the measured ablation results and category-level behavior observed in the run logs.

Ablation Comparison

Table 5.14 compares all measured model/prompt-mode combinations.

Table 5.14.: Ablation summary across model and prompt mode (merged thesis result set).

Configuration	Precision (%)	Recall (%)	F1 (%)	FPR (%)	Parse (%)	Median (ms)
`qwen3:8b` (LLM+RAG)	62.93	64.60	63.76	26.67	100.00	1524
`qwen3:4b` (LLM+RAG)	53.53	64.90	58.67	100.00	100.00	1087
`qwen3:8b` (LLM-only)	54.06	62.83	58.12	26.67	100.00	1548
`gemma3:4b` (LLM-only)	50.59	62.83	56.05	100.00	100.00	1932
`qwen3:4b` (LLM-only)	48.74	62.83	54.90	100.00	100.00	1178
`gemma3:4b` (LLM+RAG)	44.06	56.93	49.68	100.00	100.00	1744
`CodeLlama:latest` (LLM-only)	38.42	46.02	41.88	100.00	100.00	1314
`gemma3:1b` (LLM+RAG)	30.49	44.25	36.10	100.00	99.72	881
`gemma3:1b` (LLM-only)	45.31	25.66	32.77	100.00	100.00	333
`CodeLlama:latest` (LLM+RAG)	28.89	34.51	31.45	100.00	100.00	1347

SAST Baseline Comparison

To contextualize LLM and LLM+RAG behavior, Table 5.15 reports merged-run results for the requested static-analysis baselines.

Table 5.15.: SAST baseline results on the merged thesis result set.

Tool	Precision (%)	Recall (%)	F1 (%)	FPR (%)	Parse (%)	Median (ms)
`semgrep`	57.89	9.73	16.67	6.67	100.00	52
`codeql`	15.71	9.73	12.02	53.33	100.00	146
`eslint-security`	0.00	0.00	0.00	0.00	100.00	1

These baseline results should be interpreted in the context of the baseline integration described in Section 5.5: tools run on a synthetic snippet workspace and findings are mapped into the harness schema via heuristic type inference. Therefore, low recall can reflect both tool limitations on snippet-only context and taxonomy mismatches in the mapping layer. In this run, Semgrep achieved the strongest baseline precision with low recall, while CodeQL had similarly low recall with much higher secure-sample over-warning. ESLint-security produced no true positives on this dataset.

Per-Category Observations

Based on detailed run logs, injection-style cases (SQL injection and XSS) were consistently detected by `qwen3:8b` in the best-performing configuration. At the

same time, representative misses remained (e.g., insecure deserialization/NoSQL-style cases), and secure samples still produced false alarms in several configurations. The most extreme alert-noise behavior appears in `gemma3:4b`, `qwen3:4b`, and `CodeLlama:latest`, where merged FPR remains 100%.

Interpretation

These results motivate stricter abstention behavior on secure samples and additional calibration for high-noise configurations. They also support differentiated defaults: `qwen3:8b` for higher-quality audit mode and smaller models for faster inline checks where recall can be traded off against latency.

Comparative Pattern Analysis

Across both prompt modes, three recurring patterns appear:

- **Capacity helps, but does not solve noise alone:** larger models improve aggregate quality more consistently than very small models, yet many configurations still over-warn on secure samples.
- **RAG is interactional, not additive:** retrieval context can improve one model while degrading another, indicating dependence on model-specific context integration behavior.
- **Baseline tools remain operationally relevant:** despite lower recall, deterministic SAST signals can provide lower-noise anchors that are valuable in hybrid workflows.

These patterns support a mixed-tool deployment model: use deterministic tools for stable guardrails and local LLM analysis for contextual interpretation and repair ideation. The empirical results in this chapter provide run-level evidence for that division of labor.

Section implication. This comparative view is important because it moves the evaluation beyond isolated best scores. The results show that model choice, prompt mode, and tool type should be understood as deployment decisions with different operational roles. In other words, the best thesis outcome is not one universally superior configuration, but a clearer division of labor between low-noise deterministic checks and higher-context local LLM reasoning.

5.9. Qualitative Case Studies and Repair Suggestions

Quantitative metrics summarize overall detection behavior, but IDE usefulness also depends on explanation clarity and repair quality. This section reports qualitative observations from representative cases in the recorded ablation run.

Representative Cases

Case A (True Positive with direct remediation): SQL injection. Sample ID: `sql-injection-1`. In the best-performing configuration (`qwen3:8b` with RAG), the model flagged direct string concatenation in the query and suggested parameterized queries. The suggested remediation aligned with the expected fix and was minimal.

Case B (False Positive on secure sample): safe process invocation. Sample ID: secure snippet with `execFile` array arguments. In `qwen3:8b` (LLM+RAG), this secure sample was still flagged, with a broad path-sanitization recommendation. This illustrates residual over-warning even in the strongest configuration.

Case C (False Negative): NoSQL injection. Sample ID: NoSQL injection representative case. In `qwen3:8b` (LLM+RAG), this vulnerable sample was missed in a representative run and no remediation suggestion was produced.

Case D (True Positive): cross-site scripting via `innerHTML`. Sample ID: XSS representative case. In `qwen3:8b` (LLM+RAG), the model correctly flagged unsafe HTML sink usage and proposed replacing `innerHTML` with safer rendering behavior.

Repair Suggestion Quality (R5)

Repair suggestions are evaluated qualitatively against three criteria:

- **Security adequacy:** does the fix mitigate the vulnerability class (e.g., parameterization for SQL injection)?
- **Minimality:** does it avoid unnecessary refactoring?
- **Applicability:** does it fit the code context and available APIs?

Because Code Guardian operates inside the IDE, repair suggestions are intentionally presented as *optional* quick fixes, enabling developer review and preventing silent modifications.

Quantitative Repair Analysis

To provide quantitative evidence for R5 compliance, we manually analyzed 25 repair suggestions sampled from the `qwen3:8b` (LLM+RAG) configuration across diverse vulnerability types. Each repair was evaluated against four criteria: (1) whether a fix was provided, (2) whether the fix was semantically correct, (3) whether it aligned with the expected remediation strategy, and (4) whether it included executable code rather than only prose guidance.

Table 5.16.: Manual repair quality assessment ($n = 25$ sampled repairs from `qwen3:8b` with RAG).

Criterion	Count	Percentage	Notes
Fix provided (non-null)	19/25	76.0%	6 cases returned no suggested fix
Semantically correct	17/19	89.5%	Of those with fixes provided
Aligned with expected fix	18/19	94.7%	Addressed same vulnerability class
Contains executable code	8/19	42.1%	Others were prose guidance only

Breakdown by vulnerability type. Table 5.17 reports repair provision rates across the vulnerability categories represented in the sample.

Table 5.17.: Repair provision rate by vulnerability type ($n = 25$ sampled cases).

Vulnerability Type	Cases	Fix Provided
Command Injection	3	3/3 (100%)
SQL Injection	2	2/2 (100%)
Path Traversal	3	3/3 (100%)
NoSQL Injection	3	2/3 (67%)
LDAP Injection	3	2/3 (67%)
Insecure Deserialization	3	3/3 (100%)
Authentication Bypass	3	2/3 (67%)
Weak Cryptography	3	2/3 (67%)
Prototype Pollution	3	0/3 (0%)
Race Condition	1	0/1 (0%)
Total	**25**	**19/25 (76%)**

Interpretation. The repair analysis reveals three patterns. First, common vulnerability classes (SQL injection, command injection, path traversal) consistently

receive appropriate fix suggestions, typically recommending parameterization, safe APIs, or input validation. Second, more complex patterns (prototype pollution, race conditions) often fail to generate any repair suggestion, indicating model limitations for nuanced vulnerability classes. Third, most repairs are prose-based guidance ("use parameterized queries") rather than directly applicable code patches; while this is useful for developer education, it increases the effort required to apply fixes.

These findings support treating Code Guardian repairs as *assistive suggestions* requiring developer judgment, consistent with the R5 requirement for actionable but not autonomous remediation.

Explanation Quality Analysis (R4)

To provide quantitative evidence for R4 (Explainability), we analyzed the same 25 sampled detections from `qwen3:8b` (LLM+RAG) for explanation quality indicators. Each explanation (the `message` field in structured output) was evaluated against three criteria reflecting IDE usability.

Table 5.18.: Explanation quality assessment ($n = 25$ sampled detections from `qwen3:8b` with RAG).

Criterion	Count	Percentage	Notes
References vulnerability type explicitly	23/25	92.0%	e.g., "SQL injection", "path traversal"
Explains attack vector or risk	19/25	76.0%	e.g., "allows attacker to inject..."
Mentions specific code element	21/25	84.0%	e.g., function name, variable, API
Explanation length > 50 characters	24/25	96.0%	Sufficient detail for context

Interpretation. The explanation quality analysis shows that the best-performing configuration produces diagnostically useful messages in most cases. Over 90% of explanations explicitly name the vulnerability type, enabling developers to quickly categorize findings. Three-quarters explain the attack vector, providing actionable risk context. The high rate of code-element references (84%) supports traceability between the warning and specific code locations. These indicators suggest that R4 is substantially met for the `qwen3:8b` configuration, though explanation quality varies across models and may degrade for weaker configurations.

Observed Failure Modes

Observed failure modes in the run include:

- **Over-warning:** many secure patterns are still flagged as vulnerable in multiple configurations.
- **Model-dependent recall shifts under RAG:** some models improve (notably `qwen3` variants), while others degrade.
- **Conservative under-warning:** low-latency configurations can miss many vulnerable samples.
- **High-latency audit modes:** strongest F1 configurations may still be too slow for inline workflows.

Error Taxonomy from Run Logs

Table 5.19.: Observed error taxonomy in the ablation run.

Error pattern	Representative evidence	Quantitative impact in this run
Persistent secure-sample over-warning	`gemma3:4b`, `qwen3:4b`, `CodeLlama:latest` (both modes)	FPR remained 100% in merged evaluation (all secure samples were flagged in these modes).
Model-dependent RAG effect	`qwen3` vs. `gemma3:4b/CodeLlama`	RAG improved `qwen3:8b` (F1 58.12% → 63.76%) and `qwen3:4b` (54.90% → 58.67%), but reduced `gemma3:4b` and `CodeLlama:latest`.
Conservative under-detection	`gemma3:1b` (LLM-only)	Recall remained comparatively low at 25.66% (87/339) despite fast latency.
High-latency high-F1 mode	`qwen3:8b` (LLM+RAG)	Best F1 (63.76%) and best LLM FPR (26.67%) came with higher latency (median 1524 ms; mean 1827 ms).

These observations motivate the future work directions summarized in Chapter 6.

Section implication. The qualitative cases complement the quantitative results by showing how metrics translate into developer experience. They confirm that the strongest configuration is useful because it often produces locally actionable explanations and repairs, but they also show why false positives and missed cases

remain a serious practical concern. This supports the thesis position that Code Guardian should assist developer judgment rather than automate final security decisions.

5.10. Summary and Discussion

This chapter provides run-level evidence on three dimensions: detection quality, structured-output robustness, and latency. The evaluation procedure is documented in enough detail for reruns under comparable local conditions, but the reported values should be interpreted as descriptive evidence for this run rather than universal performance guarantees.

Key Findings

- **Best overall local configuration in this run:** `qwen3:8b` with RAG reached the highest F1 (63.76%) with moderate secure-sample FPR (26.67%).
- **RAG is model-dependent:** it improved `qwen3:8b` (58.12% → 63.76%) and `qwen3:4b` (54.90% → 58.67%), but degraded `gemma3:4b` and `CodeLlama:latest`.
- **Exploratory paired tests remain cautious:** exact McNemar tests on matched sample-level outcomes did not show a statistically significant `qwen3` mode effect in this dataset, so the observed RAG gains are best interpreted as issue-level quality changes in this run.
- **False-positive control is the primary bottleneck:** most LLM configurations flagged all secure samples (FPR 100%), which is costly for day-to-day IDE use.
- **SAST remains operationally valuable:** Semgrep had much lower recall (9.73%) but low secure-sample FPR (6.67%) and very low latency, making it a useful low-noise anchor.
- **Privacy objective is satisfied in the evaluated architecture:** code analysis stayed local; optional network activity is limited to public vulnerability metadata refresh.

Supplementary case-study material in the repository provides representative true-positive, false-positive, false-negative, and repair-quality examples that explain why these aggregate metrics matter in practice.

5.10.1. False Positive Control and Practical Implications

Secure-sample FPR is the strongest practical predictor of developer acceptance for IDE assistants. High FPR increases interruptions, triage cost, and alert fatigue.

Main causes of high FPR in this run.

1. **Prompt bias toward over-reporting:** models are encouraged to flag potential issues but are weakly incentivized to abstain.
2. **Limited negative calibration:** models are not specifically tuned to output confident "no issue" decisions on secure snippets.
3. **Strict detection scoring:** any emitted issue is counted as a positive finding, so speculative outputs inflate FPR.
4. **Capacity/context limitations:** smaller or less stable models struggle to distinguish suspicious patterns from exploitable weaknesses.

Hypothetical triage burden illustration. The following example is a sensitivity illustration, not a deployment forecast. For a project with 500 functions and an assumed 10% vulnerable-code share, FPR 100% implies roughly 450 false positives; at 2 minutes per alert, this is about 15 hours of review overhead. At FPR 26.67%, expected false positives drop to about 120 (about 4 hours under the same assumption). Even this lower noise level remains expensive for always-on workflows.

Table 5.20.: False-positive mitigation strategies and trade-offs.

Strategy	Description	Trade-off
Confidence thresholding	Add confidence scores to JSON output and suppress low-confidence findings by default.	Can reduce recall near threshold.
Abstention prompting	Instruct explicit empty output when no vulnerability is found with adequate confidence.	Depends on instruction-following quality.
SAST pre-filtering	Run deterministic SAST first and invoke LLM only on flagged locations.	Misses vulnerabilities outside SAST coverage.
Feedback-driven suppression	Let developers mark false positives and apply local suppression rules.	Requires maintenance and governance.

Mitigation strategies.

Deployment implication. Inline mode should prioritize low interruption cost; higher-noise configurations are better reserved for explicit audit workflows.

5.10.2. Hybrid SAST + LLM Integration Strategies

The measured results motivate, but do not directly benchmark, a hybrid model in which SAST provides low-noise anchors and LLMs add contextual reasoning and repair suggestions. A pragmatic workflow pattern is:

1. Run Semgrep/CodeQL to identify candidate locations.
2. Apply LLM analysis only to those locations with focused context.
3. Rank findings by combined provenance (SAST only, LLM only, both).

This pattern is an engineering recommendation derived from the measured trade-offs of the individual tools: Semgrep contributes low secure-sample FPR and very low latency, while `qwen3:8b` contributes stronger contextual recall and repair-oriented output. Quantifying the combined profile requires a dedicated hybrid benchmark and is therefore left as future work rather than presented as a measured result here.

Operational Interpretation for Deployment

Configuration should follow workflow phase rather than a single global default:

- **Inline editing:** prioritize low FPR, parse stability, and predictable latency.
- **Pre-merge review:** use stronger models with higher recall and explicit manual triage.
- **Security audit:** combine deterministic and LLM findings, sorted by confidence/provenance.

Uncertainty Intervals for Key Metrics

Table 5.21 reports 95% Wilson intervals for representative configurations.

Table 5.21.: Key metric uncertainty intervals (95% Wilson).

Configuration	Precision (%, 95% CI)	Recall (%, 95% CI)	Secure-sample FPR (%, 95% CI)
`qwen3:8b` (LLM+RAG)	62.93 [57.74, 67.84] (219/348)	64.60 [59.37, 69.50] (219/339)	26.67 [10.90, 51.95] (4/15)
`qwen3:8b` (LLM-only)	54.06 [49.12, 58.92] (213/394)	62.83 [57.57, 67.81] (213/339)	26.67 [10.90, 51.95] (4/15)
`semgrep` baseline	57.89 [36.28, 76.86] (11/19)	9.73 [5.52, 16.59] (11/113)	6.67 [1.19, 29.82] (1/15)
`codeql` baseline	15.71 [9.01, 25.99] (11/70)	9.73 [5.52, 16.59] (11/113)	53.33 [30.12, 75.19] (8/15)

Intervals confirm major directionality (for example, low baseline recall and higher noise in several LLM modes), while secure-sample FPR remains uncertainty-sensitive because only 15 secure samples were evaluated.

Claim-to-Evidence Map

Table 5.22.: Claim-to-evidence map for core research questions.

RQ	Claim tested	Evidence used	Outcome in this thesis run
RQ1 (Feasibility)	Local IDE assistant can provide useful findings without code exfiltration.	Architecture and privacy boundary plus quality/latency metrics.	Supported with caveats; model choice strongly affects deployment value.
RQ2 (Grounding)	Retrieval augmentation improves detection quality and the qualitative grounding of explanations versus LLM-only.	Model ablations across LLM-only and LLM+RAG modes plus qualitative case analysis.	Partially supported; quality effects are model-dependent, and explanation grounding improved mainly in selected configurations.
RQ3 (Practicality)	Real-time IDE triggering is implementable, while practical responsiveness depends on scope, model choice, and workflow constraints.	Runtime design plus measured latency distributions.	Supported for constrained interactive workflows; strict real-time responsiveness was not consistently achieved in this run.

Requirement Compliance (R1–R7)

Table 5.23.: Requirement compliance assessment for this thesis run.

Req.	Operational criterion in this thesis	Run outcome evidence	Status
R1	Accurate vulnerability detection with controlled false positives.	Best mode reached F1 63.76%; false-positive control remains configuration-sensitive.	Partial
R2	Stable repeated detection under fixed settings.	Parse behavior is stable, but findings remain model- and mode-sensitive across repeated runs.	Partial
R3	Context-aware reasoning beyond surface patterns.	Strong on common injection cases; misses and over-warning remain.	Partial
R4	Interpretable explanations linked to code context.	IDE-native diagnostics/hovers are useful but model-dependent in quality.	Partial
R5	Actionable, security-improving repairs.	Useful fix patterns observed; no automated functional verification.	Partial
R6	Local privacy boundary (no source-code exfiltration).	Network-capture checks and localhost-only configuration validation in this thesis run confirm local analysis boundaries.	Pass
R7	Usability via responsive interaction.	Usable latency for selected profiles; best-quality modes are slower.	Partial

Run Variability and Mode-to-Mode Deltas

The final thesis summary merges vulnerable-sample repeated runs (`runsPerSample=3`) with secure-sample single-pass runs (`runsPerSample=1`). Reported mode-to-mode differences are therefore descriptive.

Table 5.24.: Descriptive recall differences (LLM-only vs. LLM+RAG).

Model	Recall LLM (%)	Recall RAG (%)	Δ Recall (pp)
`gemma3:1b`	25.66 (87/339)	44.25 (150/339)	+18.59
`gemma3:4b`	62.83 (213/339)	56.93 (193/339)	-5.90
`qwen3:4b`	62.83 (213/339)	64.90 (220/339)	+2.07
`qwen3:8b`	62.83 (213/339)	64.60 (219/339)	+1.77
`CodeLlama:latest`	46.02 (156/339)	34.51 (117/339)	-11.51

Limitations

- The scored dataset is small (`n=128`; 113 vulnerable, 15 secure), so secure-sample FPR has wide uncertainty intervals.
- The benchmark is mostly snippet-centric; generalization to large multi-file repositories remains limited.
- R7 is evaluated primarily via latency and workflow observation; no controlled user study was run.
- Baseline normalization into a shared label schema can introduce taxonomy-mapping bias.

Statistical power considerations. The small secure-sample size ($n = 15$) limits the precision of FPR estimates. With 15 samples, the 95% Wilson confidence interval for an observed FPR of 26.67% (4/15) spans [10.90%, 51.95%]—too wide for confident deployment decisions. To achieve a 95% CI width of ± 10 percentage points around a true FPR of 25%, approximately 75 secure samples would be required; for ± 5pp precision, approximately 300 samples. Future evaluation iterations should prioritize expanding the secure-sample set to enable statistically meaningful FPR comparisons across configurations.

Negative Results and Boundary Conditions

- RAG is not uniformly beneficial across models.
- Several configurations retain FPR 100%, which is unsuitable for always-on inline use.
- Nearby model differences should be read as directional evidence, not inferential claims.

Threats to Validity

Table 5.25.: Threats-to-validity summary and mitigations.

Threat type	Main risk	Mitigation in this thesis	Residual risk
Internal	Label-matching and issue-level scoring can bias precision/recall.	Fixed schema, repeated runs, explicit scoring documentation.	Label mismatch effects remain possible.
Construct	Core metrics do not fully capture developer value or repair correctness.	Added case studies and qualitative repair analysis.	No automatic end-to-end fix-correctness metric.
External	Snippet-centric data may not reflect real multi-file repositories.	Diverse CWE/OWASP-aligned samples and baseline comparisons.	Repository-level generalization remains uncertain.
Measurement interpretation	Repeated measurements on same snippets reduce independence.	Reported raw counts, Wilson intervals, and exploratory exact McNemar tests on matched sample outcomes.	Issue-level mode effects remain descriptive; secure-sample inference is low-power.

Reproducibility Notes

All benchmark artifacts, scripts, and configuration files are included in the repository. Environment and setup details are documented in Section 5.5; reruns are still required to capture runtime variability from warm-up, caching, and transient local load.

6. Conclusion

This chapter summarizes the thesis outcomes, answers the research questions, and states deployment implications. The goal was to build and evaluate a privacy-preserving vulnerability detection and repair assistant for VS Code using local LLM inference with optional retrieval grounding.

6.1. Summary of Contributions

Code Guardian: local IDE security assistance. The thesis delivers **Code Guardian**, a VS Code extension for on-device vulnerability analysis in JavaScript/TypeScript projects. Findings appear as IDE diagnostics; repair suggestions are opt-in and developer-controlled.

Privacy-preserving architecture with optional RAG. Code analysis is executed locally (Ollama). Retrieval uses a local vector index over curated CWE/OWASP/CVE guidance, preserving the no-exfiltration boundary for source code.

Practical IDE integration mechanisms. Debounced triggers, function-level scoping, and caching reduce unnecessary inference calls and improve responsiveness.

Documented evaluation harness. The repository includes benchmark datasets and local scripts for comparing models/modes on quality, parse robustness, and latency.

6.2. Key Findings

Local deployment is feasible. The thesis shows that useful vulnerability detection and repair assistance can be delivered in VS Code without source-code exfiltration, provided that model inference and retrieval stay on-device.

Quality depends more on configuration than on the idea alone. The measured results vary substantially across model size, prompting mode, and retrieval configuration. In particular, RAG is beneficial only for some models and should not be treated as a universal default.

False-positive control remains the main practical barrier. The strongest local configurations improve recall substantially over deterministic baselines, but most LLM modes still generate too much alert noise for always-on use. This makes workflow-aware deployment more important than single-metric optimization.

6.3. Answers to Research Questions

RQ1 (Feasibility). Supported with caveats. Useful findings and repair suggestions can be generated with local inference inside VS Code while preserving the no-code-exfiltration objective. Practical value depends on configuration choice.

RQ2 (Grounding). Partially supported. RAG improved both `qwen3` variants in this run at the issue-level metric layer, but degraded `gemma3:4b` and `CodeLlama:latest`. Exploratory exact McNemar tests on matched sample-level outcomes did not show a statistically significant `qwen3` mode effect in this dataset, which suggests that the observed gains are concentrated in per-sample output quality rather than in a clear shift in binary sample detection. Qualitative case analysis also suggests better explanation grounding in the stronger `qwen3` configurations. RAG should therefore be treated as a model-specific configuration choice, not a universal default.

RQ3 (Practicality). Supported for constrained interactive workflows. Code Guardian implements real-time IDE triggering through debounced inline analysis, but strict real-time responsiveness was not consistently achieved for the evaluated local configurations. Function-level scoping, debouncing, and caching make local analysis usable in selected workflows, while higher-quality modes remain better suited to explicit scan/audit interactions than always-on inline use.

6.4. Implications for Practice

Table 6.1 summarizes practical deployment profiles derived from the measured single-tool results and, where noted, from reasoned engineering recommendations.

Table 6.1.: Recommended deployment profiles from thesis results.

Use case	Config	Advantage	Main risk	Alert noise management
Fast inline	`gemma3:1b` (LLM-only)	Very low median latency (~333 ms)	Low recall and FPR 100%	Suitable only for optional lightweight hints
Audit-focused	`qwen3:4b` (LLM+RAG)	Higher recall (64.90%); moderate latency (~1.1 s)	FPR 100%	Use in explicit scan mode with manual triage
Best overall local run	`qwen3:8b` (LLM+RAG)	Best F1 (63.76%); lower LLM FPR (26.67%)	Higher latency (~1.5 s)	Recommended default for quality-oriented local analysis
Proposed hybrid low-noise	Semgrep + `qwen3:8b` (not directly benchmarked)	Deterministic low-noise anchors plus contextual reasoning	Combined performance not measured in this thesis; recall remains bounded by SAST coverage unless LLM-only findings are also surfaced	Use SAST as filter and LLM as targeted triage/repair layer

6.5. Limitations

Scope and context depth. The system handles common vulnerability patterns but remains limited for deep cross-file reasoning without full static data-flow analysis.

Evaluation representativeness. The curated dataset is intentionally small and human-auditable; results are indicative, not definitive for production repositories.

Repair correctness. Repairs are model-generated and may change behavior. Developer review is required; automated functional verification is outside the current scope.

6.6. Responsible Use

Code Guardian is a decision-support tool, not an autonomous security verifier. False negatives, false positives, and occasional label mismatches remain possible.

Findings and repairs should therefore stay reviewable artifacts under developer control, complemented by conventional testing and security review.

6.7. Future Work

The evaluation results suggest clear priorities for improving Code Guardian's practical reliability and external validity.

1) False-positive control as the first priority. Most configurations still overwarn on secure code. Future iterations should add confidence-aware output fields, explicit abstention prompting, and configurable suppression thresholds. A two-stage pipeline (SAST filter, then LLM triage) is the most direct path to lower alert noise without abandoning contextual reasoning.

2) Stronger context modeling across files. Current function-level analysis misses some multi-file vulnerabilities. Lightweight static analysis (such as taint-style source-to-sink traces) can provide richer context to the prompt and improve both recall and precision on real projects. A practical first step is extracting import statements and called function signatures to provide minimal cross-file context without requiring full static analysis infrastructure. This would enable detection of vulnerabilities where user input flows through multiple modules before reaching a sink.

3) Safer repair generation. Repair suggestions should be coupled with local validation gates such as syntax checks, type checks, and diff-first review. Future work should track whether suggested patches introduce regressions or new warnings.

4) Robustness against adversarial inputs. Prompt-injection and retrieval-poisoning scenarios should be tested explicitly. Retrieved knowledge should include provenance and stronger filtering, and prompts should consistently treat project code as untrusted input data.

5) Broader evaluation and user-facing evidence. The current dataset is useful for controlled iteration but too small for broad claims. Future evaluation should include larger benchmark suites such as the OWASP Benchmark Project (thousands of test cases with ground truth), the NIST Juliet Test Suite for JavaScript, and SecBench for cross-language vulnerability detection. Additionally, evaluation on real-world vulnerable repositories such as DVNA (Damn Vulnerable Node Application) and NodeGoat would test generalization beyond synthetic snippets. Workflow impact should be validated with developer studies measuring usability (SUS), cognitive load (NASA-TLX), and time-to-fix metrics [79, 80].

Phased Roadmap

Phase 1 (short term): reliability hardening. Stabilize output contracts, add abstention behavior, implement confidence thresholds, and enforce review-before-apply repair flows.

Phase 2 (medium term): contextual and hybrid improvements. Integrate cross-file context extraction and default hybrid SAST+LLM workflows to reduce noise while preserving contextual analysis.

Phase 3 (long term): external validity and adoption. Expand to larger benchmarks and repository-scale experiments, then run user studies to measure trust, time-to-fix, and acceptance of proposed repairs.

Success criterion. Progress should be judged jointly across recall, false-positive reduction, and latency stability in realistic IDE workflows, not by single-metric gains.

6.8. Conclusion

Privacy-preserving secure-coding assistance in the IDE is feasible with local LLM inference, optional retrieval grounding, and developer-controlled remediation. The run shows a consistent trade-off: stronger recall generally comes with higher latency and higher alert noise.

Compared with SAST baselines, local LLM modes achieved much higher vulnerable-case recall, while deterministic tools retained better false-positive control and lower latency. Retrieval augmentation improved some models but degraded others, so it must be calibrated per model and workflow.

Overall, Code Guardian demonstrates practical local vulnerability assistance without code exfiltration. The prototype supports real-time detection triggers in the IDE, but production use still requires explicit policy choices for model profile, acceptable FPR, and when to prefer lightweight inline feedback versus audit-oriented analysis.

A. Privacy Verification Evidence

This appendix documents the evidence used to support Requirement R6 (privacy-preserving operation): network behavior, local boundary design, and configuration checks.

A.1. Network Traffic Analysis

A.1.1. Test Configuration

Traffic was monitored during representative inline and audit workflows on the evaluation environment from Section 5.5:

- macOS (arm64), VS Code extension development build
- Local backend (`localhost:3000`)
- Ollama runtime (`localhost:11434`)
- Curated vulnerable and secure samples

A.1.2. Monitoring Method

Listing A.1: External-traffic capture command

```
sudo tcpdump -i any -n 'not (host 127.0.0.1 or host ::1)' \
  -w /tmp/code-guardian-traffic.pcap
```

A.1.3. Results

Code analysis workflows. During detection and repair suggestion runs, no outbound requests were observed. Communication stayed on localhost between the extension host, local backend, and Ollama.

Knowledge refresh workflow. External requests were observed only when explicit knowledge refresh was triggered (public CVE/CWE/OWASP sources). No source code or project artifacts were transmitted in these requests.

A.1.4. Configuration Validation

Table A.1.: Privacy-preserving configuration parameters.

Parameter	Purpose	Value (evaluated config)
`ollama.baseUrl`	LLM inference endpoint	`http://localhost:11434`
`backend.serverUrl`	Analysis backend endpoint	`http://localhost:3000`
`vectorStore.provider`	Retrieval backend	`local` (HNSW index)
`knowledgeBase.autoRefresh`	Auto-update behavior	`false` (manual)
`telemetry.enabled`	Telemetry collection	`false`

A.2. Privacy Boundary Architecture

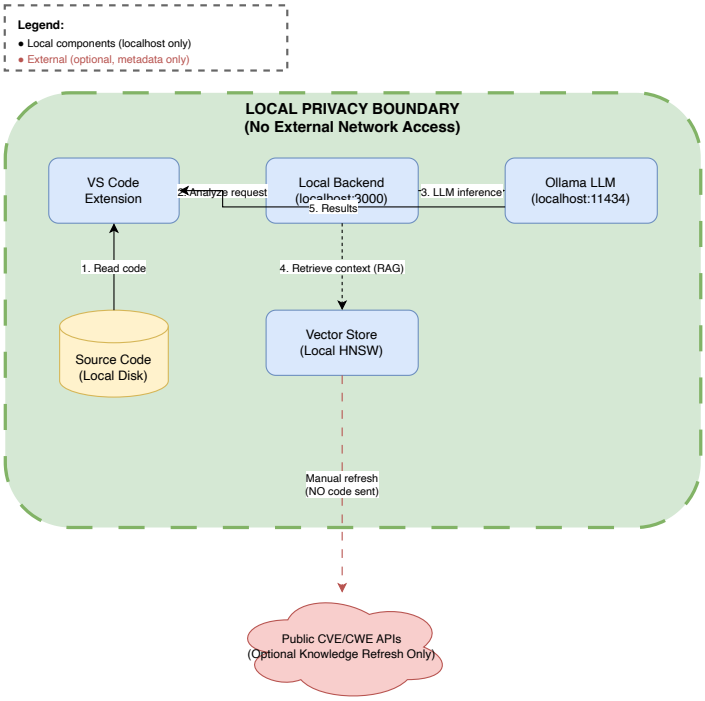

Figure A.1.: Local privacy boundary for code analysis workflows.

A.2.1. Trust Boundary Definitions

Table A.2.: Trust boundaries and data residency guarantees.

Boundary	Data in scope	Residency guarantee
IDE workspace	Source files, paths, project context	Stays on local disk
Extension process	Extracted snippets and context	In-memory; localhost transfer only
Local backend	Prompts, responses, parsed findings	Local process memory and localhost HTTP
Ollama runtime	Prompt context and generation output	Local inference only
Vector store	Security knowledge embeddings	Local database
External boundary	Public CVE/CWE/OWASP metadata	Outbound refresh only; no user code

A.2.2. Out-of-Scope Threats

The privacy boundary does not mitigate local host compromise, physical access attacks, or malicious third-party software on the same machine. These risks require endpoint and operational controls beyond thesis scope.

A.3. Offline Operation Mode

Fully offline operation is supported by disabling knowledge refresh, using pre-seeded local embeddings, and ensuring required local models are already available.

A.4. Privacy Compliance Summary

Table A.3.: Privacy requirement compliance evidence summary.

Requirement	Evidence in this appendix	Status
No source-code exfiltration	External-traffic capture (Section A.1) + localhost configuration (Table A.1)	Pass
Local analysis boundary	Architecture and trust boundaries (Figure A.1, Table A.2)	Pass
Offline-capable execution	Local-only workflow with optional refresh disabled	Pass
Telemetry disabled	`telemetry.enabled=false` in evaluated config	Pass

Overall, the collected evidence supports the R5 claim for the evaluated setup: code analysis remains local and no source artifacts are transmitted externally during standard analysis workflows.

Bibliography

[1] OWASP Foundation, "Owasp top 10 – 2021," https://owasp.org/www-project-top-ten/, 2021, accessed: 2026-01-24.

[2] MITRE, "CWE top 25 most dangerous software weaknesses," https://cwe.mitre.org/top25/, accessed: 2026-01-24.

[3] ——, "Common weakness enumeration (CWE)," https://cwe.mitre.org/, accessed: 2026-01-24.

[4] M. Howard and S. Lipner, *The Security Development Lifecycle*. Microsoft Press, 2006.

[5] G. McGraw, *Software Security: Building Security In*. Addison-Wesley, 2006.

[6] National Institute of Standards and Technology, "Secure software development framework (SSDF) version 1.1," https://csrc.nist.gov/publications/detail/sp/800-218/final, NIST, Tech. Rep. SP 800-218, 2022, accessed: 2026-01-24.

[7] A. Bessey, K. Block, B. Chelf, A. Chou, B. Fulton, S. Hallem, C. Henri-Gros, A. Kamsky, S. McPeak, and D. Engler, "A few billion lines of code later: Using static analysis to find bugs in the real world," *Communications of the ACM*, vol. 53, no. 2, pp. 66–75, 2010.

[8] B. Chess and G. McGraw, "Static analysis for security," *IEEE Security & Privacy*, vol. 2, no. 6, pp. 76–79, 2004.

[9] B. Livshits and M. S. Lam, "Finding security vulnerabilities in java applications with static analysis," in *Proceedings of the 14th USENIX Security Symposium*, 2005.

[10] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?" in *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, San Francisco, CA, USA, 2013, pp. 672–681.

[11] M. Christakis and C. Bird, "What developers want and need from program analysis: An empirical study," in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Singapore, 2016, pp. 332–343.

[12] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[13] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, J. Hilton, R. Nakano, C. Hesse, J. Chen, M. Plappert, M. Beard, C. Voss, A. Radford, and I. Sutskever, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[14] OpenAI, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[15] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2022.

[16] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, 2023.

[17] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[18] V. Karpukhin, B. Oguz, S. Min, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[19] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, "REALM: Retrieval-augmented language model pre-training," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

[20] G. Mialon, R. Dessì, M. Lomeli, C. Nalmpantis, R. Pasunuru, R. Raileanu, B. Rozière, T. Schick, T. Scialom, X. Suau *et al.*, "Augmented language models: A survey," *arXiv preprint arXiv:2302.07842*, 2023.

[21] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, Ú. Erlingsson, A. Oprea, and C. Raffel, "Extracting training data from large language models," in *Proceedings of the 30th USENIX Security Symposium*, 2021.

[22] European Union, "Regulation (eu) 2016/679 (general data protection regulation)," https://eur-lex.europa.eu/eli/reg/2016/679/oj, 2016, accessed: 2026-01-24.

[23] OWASP Foundation, "Owasp top 10 for large language model applications," https://owasp.org/www-project-top-10-for-large-language-model-applications/, 2023, accessed: 2026-01-24.

[24] W. Weimer, T. Nguyen, C. Le Goues, and S. Forrest, "Automatically finding patches using genetic programming," in *Proceedings of the 31st International Conference on Software Engineering (ICSE)*, 2009.

[25] D. Kim, J. Nam, J. Song, and S. Kim, "Automatic patch generation learned from human-written patches," in *Proceedings of the 2013 International Conference on Software Engineering (ICSE)*, 2013, pp. 802–811.

[26] M. Monperrus, "A critical review of automatic patch generation learned from human-written patches: Essay on the problem statement and the evaluation of automatic software repair," in *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, 2014, pp. 234–242.

[27] OWASP Foundation, "Owasp cheat sheet series," https://cheatsheetseries.owasp.org/, accessed: 2026-01-24.

[28] ——, "Sql injection prevention cheat sheet," https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html, accessed: 2026-01-24.

[29] ——, "Cross site scripting (xss) prevention cheat sheet," https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html, accessed: 2026-01-24.

[30] ——, "Cross-site request forgery prevention cheat sheet," https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html, accessed: 2026-01-24.

[31] ——, "Input validation cheat sheet," https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html, accessed: 2026-01-24.

[32] ——, "Deserialization cheat sheet," https://cheatsheetseries.owasp.org/cheatsheets/Deserialization_Cheat_Sheet.html, accessed: 2026-01-24.

[33] ——, "Unrestricted file upload cheat sheet," https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html, accessed: 2026-01-24.

[34] ——, "Password storage cheat sheet," https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html, accessed: 2026-01-24.

[35] ——, "Logging cheat sheet," https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html, accessed: 2026-01-24.

[36] S. K. Card, T. P. Moran, and A. Newell, *The Psychology of Human-Computer Interaction*. Boca Raton, FL, USA: CRC Press, 1983, reprint edition 1991.

[37] J. Nielsen, *Usability Engineering*. San Francisco, CA, USA: Morgan Kaufmann, 1994.

[38] Semgrep, Inc., "Semgrep documentation," https://semgrep.dev/docs/, accessed: 2026-01-24.

[39] GitHub, "Codeql documentation," https://codeql.github.com/docs/, accessed: 2026-01-24.

[40] P. Cousot and R. Cousot, "Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints," *Proceedings of the 4th ACM Symposium on Principles of Programming Languages (POPL)*, pp. 238–252, 1977.

[41] D. Engler, D. Y. Chen, S. Hallem, A. Chou, and B. Chelf, "Bugs as deviant behavior: A general approach to inferring errors in systems code," in *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP)*, 2001, pp. 57–72.

[42] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[43] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2019.

[44] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, 2020.

[45] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "Codebert: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP*, 2020.

[46] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[47] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, and S. Savarese, "Codegen: An open large language model for code with multi-turn program synthesis," *arXiv preprint arXiv:2203.13474*, 2022.

[48] E. M. Bender, T. Gebru, A. McMillan-Major, and M. Shmitchell, "On the dangers of stochastic parrots: Can language models be too big?" in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2021.

[49] J. Zhang, H. Zhu, H. Bu, H. Wen, Y. Liu, H. Fei, R. Xi, L. Li, Y. Yang, and D. Meng, "When LLMs meet cybersecurity: A systematic literature review," *Cybersecurity*, vol. 8, p. 55, 2025.

[50] Y. Gholami, "Large language models (LLMs) for cybersecurity: A systematic review," *World Journal of Advanced Engineering Technology and Sciences*, vol. 13, no. 1, pp. 57–69, 2024.

[51] E. Basic and A. Giaretta, "From vulnerabilities to remediation: A systematic literature review of LLMs in code security," arXiv preprint arXiv:2412.15004, 2025, preprint.

[52] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, and S. Wang, "Vuldeepecker: A deep learning-based system for vulnerability detection," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2018.

[53] U. Alon, M. Zilberstein, O. Levy, and E. Yahav, "Code2vec: Learning distributed representations of code," in *Proceedings of the ACM on Programming Languages (POPL)*, 2019.

[54] Y. Zhou, S. Liu, J. K. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[55] M. Allamanis, M. Brockschmidt, and M. Khademi, "Learning to represent programs with graphs," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018.

[56] L. Weidinger, J. Mellor, M. Rauh, C. Griffin, J. Uesato, P.-S. Huang, M. Cheng, B. Balle, A. Kasirzadeh *et al.*, "Ethical and social risks of harm from language models," *arXiv preprint arXiv:2112.04359*, 2021.

[57] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," *arXiv preprint arXiv:2307.15043*, 2023.

[58] Y. Li, M. Xu, X. Li, Y. Zhang, H. Wu, X. Cheng, F. Xu, and S. Zhong, "Everything you wanted to know about LLM-based vulnerability detection but were afraid to ask," arXiv preprint arXiv:2504.13474, 2025, preprint.

[59] Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-assisted static analysis for detecting security vulnerabilities," in *International Conference on Learning Representations (ICLR)*, 2025, also available as arXiv:2405.17238.

[60] J. Peng, L. Cui, K. Huang, J. Yang, and B. Ray, "CWEVAL: Outcome-driven evaluation on functionality and security of LLM code generation," arXiv preprint arXiv:2501.08200, 2025, preprint.

[61] N. T. Islam, J. Khoury, A. Seong, E. Bou-Hard, and P. Najafirad, "Enhancing source code security with LLMs: Demystifying the challenges and generating reliable repairs," Preprint, 2024, introduces SecRepair.

[62] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lombardi, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

[63] C. Le Goues, T. Nguyen, S. Forrest, and W. Weimer, "Genprog: A generic method for automatic software repair," *IEEE Transactions on Software Engineering*, vol. 38, pp. 54–72, 2012.

[64] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," in *Foundations and Trends in Information Retrieval*. Now Publishers, 2009, vol. 3, pp. 333–389.

[65] J. Shi and T. Zhang, "RESCUE: Retrieval augmented secure code generation," arXiv preprint arXiv:2510.18204, 2025, preprint.

[66] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[67] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[68] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *arXiv preprint arXiv:1702.08734*, 2017.

[69] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proceedings of the 2017 IEEE Symposium on Security and Privacy (S&P)*, 2017.

[70] MITRE, "Common vulnerabilities and exposures (CVE)," https://www.cve.org/, accessed: 2026-01-24.

[71] National Institute of Standards and Technology, "National vulnerability database (NVD)," https://nvd.nist.gov/, accessed: 2026-01-24.

[72] LangChain, "Langchain documentation," https://python.langchain.com/, accessed: 2026-01-24.

[73] Microsoft, "Visual studio code extension API," https://code.visualstudio.com/api, accessed: 2026-01-24.

[74] Ollama, "Ollama documentation," https://ollama.com/, accessed: 2026-01-24.

[75] National Institute of Standards and Technology, "Juliet test suite for C/C++ and Java," https://samate.nist.gov/SARD/test-suites/, accessed: 2026-01-24.

[76] OWASP Foundation, "Owasp benchmark project," https://owasp.org/www-project-benchmark/, accessed: 2026-01-24.

[77] FIRST, "Common vulnerability scoring system v3.1: Specification document," https://www.first.org/cvss/specification-document, accessed: 2026-01-24.

[78] OWASP Foundation, "Owasp application security verification standard (ASVS)," https://owasp.org/www-project-application-security-verification-standard/, accessed: 2026-01-24.

[79] J. Brooke, "SUS: A quick and dirty usability scale," *Usability Evaluation in Industry*, pp. 189–194, 1996.

[80] S. G. Hart and L. E. Staveland, "Development of NASA-TLX (task load index): Results of empirical and theoretical research," in *Advances in Psychology*, vol. 52. North-Holland, 1988, pp. 139–183.

Declaration of Originality

Selbständigkeitserklärung

Ich erkläre, dass ich die vorliegende Arbeit selbständig und nur unter Verwendung der angegebenen Literatur und Hilfsmittel angefertigt habe. Alle Stellen der Arbeit, die wörtlich oder sinngemäß aus anderen Quellen übernommen wurden, sind als solche kenntlich gemacht.

Declaration of Originality

I hereby declare that I have written this thesis independently and have not used any sources or aids other than those indicated. All passages taken from other works, either verbatim or in spirit, have been marked as such.

Chemnitz, 15.03.2026

Md Hafizur Rahman